



Universität der Bundeswehr München
Fakultät für Luft- und Raumfahrttechnik
Institut für autonomes Fahren

Ressourceneffiziente BEV-basierte latente Weltmodelle für das autonome Fahren

Vincent Mann, B.Sc.

Masterarbeit

Betreuer: Leon Pohl, M.Sc.

Prüfer: Prof. Dr.-Ing. Mirko Mählich
Dr.-Ing. Thorsten Lüttel

Abgabedatum: 23. August 2026

Hiermit erkläre ich, Vincent Mann, B.Sc., an Eides statt, dass ich die vorliegende Masterarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Large Language Models wurden unterstützend für Grammatik, Rechtschreibung und Programmierung eingesetzt.

Neubiberg, den 23. August 2026

Aufgabenstellung

Bird's-Eye-View (BEV) basierte Weltmodelle gewinnen zunehmend an Bedeutung für autonome Fahrfunktionen, da sie die Umgebung in einer kompakten und geometrisch konsistenten Repräsentation erfassen und deren zeitliche Dynamik modellieren können. Insbesondere latente Weltmodelle ermöglichen eine effiziente Vorhersage zukünftiger Szenenzustände direkt im Repräsentationsraum und bilden damit eine Grundlage für Planung, Entscheidungsfindung und Simulation.

Aktuelle Ansätze setzen jedoch häufig auf große und rechenintensive Architekturen, die hohe Anforderungen an Speicher, Trainingszeit und Inferenzhardware stellen. Für den praktischen Einsatz auf eingebetteten Systemen oder ressourcenlimitierten Plattformen ist daher die Entwicklung ressourceneffizienter Weltmodelle von zentraler Bedeutung.

Ziel dieser Arbeit ist die systematische Untersuchung architektonischer und trainingsbezogener Designentscheidungen zur Reduktion des Rechen- und Speicherbedarfs latenter BEV-Weltmodelle bei gleichzeitiger Erhaltung der Vorhersagequalität.

Bearbeitungsumfang

1. **Literaturrecherche** zu latenten BEV-Weltmodellen mit Fokus auf architektonische Effizienz.
2. **Implementierung und Analyse** einer Baseline-Architektur für ein BEV-basiertes latentes Weltmodell.
3. **Entwicklung und Untersuchung** ressourceneffizienter Architekturvarianten (z. B. reduzierte Modellgröße, kompakte Latentdimension).
4. **Vergleichende Evaluation** der Modelle hinsichtlich Vorhersagequalität, Trainingsaufwand, Speicherbedarf und Inferenzzeit.

Kurzfassung

Untersucht wird die Vorhersage zukünftiger Zustände in Latenträumen der Bird's Eye View (BEV)-Darstellung für Segmentierung und 3D-Objektdetektion. Ein Transformer mit rund sechs Millionen Parametern sagt aus drei aufeinanderfolgenden Zuständen im jeweiligen BEV-Latentraum den nächsten Zustand vorher. Auf dem nuScenes-Datensatz werden die Vorhersagen mit den vorhandenen Wahrnehmungsköpfen ausgewertet. Als Referenzen dienen eine ego-kompensierte Persistenz und die reale Zielrepräsentation.

Für die Bewertung werden die mean Intersection over Union (mIoU) für die Karten-segmentierung und die mean Average Precision (mAP) für die 3D-Objektdetektion verwendet. In der Kartensegmentierung genügen ein Smooth-L1-Term und ein Term für die Streuung der Repräsentationen. Zusätzliche Loss-Terme verbessern die mIoU nicht messbar, während die vollständige Sechs-Term-Zielfunktion die mAP in zwei Trainingsläufen erhöht. Die geeignete Zielfunktion hängt in diesem Versuchsaufbau somit von der nachgelagerten Aufgabe ab.

Gegen die nuScenes-Annotation erreicht das Weltmodell 0,5898 mIoU in der Segmentierung und 0,3438 mAP in der Detektion. Mit der realen Zielrepräsentation liegen die Referenzwerte bei 0,6295 mIoU beziehungsweise 0,6858 mAP. Durch die Anpassung der nachgelagerten Köpfe steigen die Werte auf $0,6008 \pm 0,0005$ mIoU und $0,4314 \pm 0,0021$ mAP. Dabei bleibt das Weltmodell unverändert.

Auf einer TITAN RTX benötigt ein Vorhersageschritt rund 4,4 ms für die Segmentierung und 11,5 ms für die Detektion. Eine größere Transformer-Variante erhöht den Ressourcenbedarf, ohne die Segmentierungsqualität messbar zu verbessern. Im untersuchten Größenbereich ist daher kein Vorteil durch zusätzliche Modellkapazität erkennbar. Zielfunktion und nachgelagerte Köpfe müssen für Segmentierung und Detektion getrennt abgestimmt werden.

Abstract

This thesis investigates the prediction of future Bird's Eye View (BEV) latents for segmentation and 3D object detection. A transformer with roughly six million parameters predicts the next latent from three consecutive BEV latents of the respective frozen BEVFusion configuration. On the nuScenes dataset, the predictions are evaluated with the existing perception heads. An ego-compensated persistence baseline and the real target latent serve as references.

The evaluation uses mean Intersection over Union (mIoU) for map segmentation and mean Average Precision (mAP) for 3D object detection. For map segmentation, a Smooth-L1 term and a term matching the latent spread are sufficient. Additional loss terms do not measurably improve mIoU, whereas the full six-term objective increases mAP in 3D object detection in two training runs. In this experimental setup, the suitable objective therefore depends on the downstream task.

Against the nuScenes annotations, the world model reaches 0,5898 mIoU for segmentation and 0,3438 mAP for detection. Using the real target latent yields reference values of 0,6295 mIoU and 0,6858 mAP, respectively. Adapting the downstream heads increases the results to $0,6008 \pm 0,0005$ mIoU and $0,4314 \pm 0,0021$ mAP while keeping the world model unchanged.

On a TITAN RTX, one prediction step takes approximately 4,4 ms for segmentation and 11,5 ms for detection. A larger transformer variant increases the computational cost without measurably improving segmentation quality. No benefit from additional model capacity is observed within the investigated range. The training objective and downstream heads therefore need to be adapted separately for segmentation and detection.

Inhaltsverzeichnis

Abkürzungsverzeichnis	vii
Abbildungsverzeichnis	ix
Tabellenverzeichnis	xi
1 Einleitung	1
1.1 Motivation und Zielsetzung	1
1.2 Forschungsfragen	2
1.3 Aufbau der Arbeit	2
2 Grundlagen und verwandte Arbeiten	3
2.1 Transformer	3
2.2 Generative Modellierung	5
2.3 BEVFusion	6
2.4 Weltmodelle und Feature-Forecasting	6
3 Konzept	9
3.1 Anforderungen und Bewertungskriterien	9
3.2 Entwurfsentscheidungen	10
3.3 Messkette und Untersuchungsplan	11
4 Implementierung	13
4.1 Systemüberblick und Datenpipeline	13
4.2 Prädiktionsmodell	14
4.3 Zielfunktion und Training	16
4.4 Messkette	17
4.5 Erweiterungen und Versuchsdokumentation	19
5 Evaluierung der Vorhersage von BEV-Repräsentationen	21
5.1 Referenzwerte und Seed-Variabilität	21
5.2 Loss-Ablation und Modellvarianten	22
5.3 Generative Vorhersageköpfe	26
5.4 Bewertung gegen die nuScenes-Annotation	28
5.5 Ressourcenbedarf	34
5.6 Empfohlene Konfigurationen und Limitationen	35
6 Zusammenfassung und Ausblick	37
A Anhang	39
A.1 Gesamtübersicht der Weltmodell-Varianten	39
A.2 Kopfvarianten beider Stränge	40
A.3 Einzelverläufe der Robustheitsprüfung	40
A.4 Ergänzungen zu den generativen Köpfen	44
A.5 Qualitative Detektionspanels	45

Literaturverzeichnis

49

Abkürzungsverzeichnis

BEV	Bird's Eye View
CVAE	Conditional Variational Autoencoder
EMA	Exponential Moving Average
FiLM	Feature-wise Linear Modulation
GELU	Gaussian Error Linear Unit
GPU	Grafikprozessor, engl. <i>Graphics processing unit</i>
GT	Ground Truth
IoU	Intersection over Union, dt. <i>Jaccard-Koeffizient</i>
JEPA	Joint-Embedding Predictive Architecture
LiDAR	Light Detection And Ranging
LN	Layer Normalization
mAP	mean Average Precision
mIoU	mean Intersection over Union
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NDS	nuScenes Detection Score
Radar	Radio Detection And Ranging
SSIM	Structural Similarity Index
VRAM	Video Random Access Memory

Abbildungsverzeichnis

2.1	Transformer-Block in Post-LN- und Pre-LN-Anordnung	4
2.2	Drei Familien generativer Modelle im Vergleich	5
2.3	BEV-Wahrnehmungspipeline mit Schnittstelle im BEV-Latentraum .	6
3.1	Gesamtsystem der Arbeit	12
4.1	Aufbau des Prädiktors	13
5.1	Referenzwerte beider Stränge	21
5.2	Einfluss der Loss-Gewichte auf der Proxy-Skala	22
5.3	Streuungsverhältnis vor und nach dem Smooth-L1-Wechsel	23
5.4	Untersuchte Modellvarianten	24
5.5	Autoregressiver Rollout in einer statischen Szene	25
5.6	Autoregressiver Rollout in einer dynamischen Szene	25
5.7	Gelernte Gate-Karten für drei Beispielszenen	26
5.8	mIoU der Minimal-Konfiguration je Klasse	26
5.9	CVAE-Kopf: Mittelwert und Stichproben	27
5.10	Flow-Matching-Kopf: Variation über die Stichproben	28
5.11	Residual-Skalierung des Flow-Kopfes	28
5.12	Detektionsstrang auf der Referenzskala	29
5.13	Segmentierungsstrang gegen die nuScenes-Annotation	30
5.14	Metrikabstände auf der Referenzskala	30
5.15	Adaptation des Segmentierungskopfs	32
5.16	Kopfversionen des Detektionsstrangs im Vergleich	32
5.17	Detektionen mit eingefrorenem und adaptiertem Kopf	33
5.18	Vorhersagequalität gegen Ressourcenbedarf	35
A.1	Rollout-Auswertung über vier Schritte	41
A.2	Ego-Konditionierung: vier Injektionsvarianten	41
A.3	Naive und ego-kompensierte Persistenz im Rollout	42
A.4	Kontext- und EMA-Varianten	42
A.5	Task-Loss, Tiefe und Residualvariante	43
A.6	CVAE-Darstellung für Beta 0,001	44
A.7	CVAE-Darstellung für Beta 0,1	45
A.8	Rollout-Vergleich: deterministisch gegen Flow	45
A.9	3D-Boxen: Weltmodell gegen Persistenz	46
A.10	3D-Boxen: eingefrorener gegen adaptierten Kopf	47
A.11	3D-Boxen: beide Fassungen des adaptierten Kopfes	48

Tabellenverzeichnis

4.1	Architekturparameter des Prädiktors	15
5.1	Bezugsgrößen der Evaluierung	21
5.2	Ressourcenbedarf der Modellvarianten	34
A.1	Weltmodell-Varianten des Segmentierungsstrangs	39
A.2	Kopfvarianten beider Stränge	40

1 Einleitung

1.1 Motivation und Zielsetzung

Ein automatisiertes Fahrzeug muss erkennen, was sich in seiner Umgebung befindet, und abschätzen, wie sich diese Umgebung in naher Zukunft verändert, um vorausschauend planen zu können. Dafür ist eine Vorhersage zukünftiger Szenenzustände erforderlich.

Modelle zur Vorhersage solcher Zustände werden als Weltmodelle bezeichnet [1] und können auf unterschiedlichen Repräsentationen arbeiten. In dieser Arbeit erfolgt die Vorhersage direkt im Merkmalsraum eines Wahrnehmungsmodells.

Große generative Weltmodelle können einen hohen Speicher- und Rechenbedarf haben. Das generative Fahrmodell GAIA-2 umfasst beispielsweise rund 8,4 Milliarden Parameter [2]. Für die hier betrachteten Wahrnehmungsaufgaben ist jedoch keine fotorealistische Zukunftsdarstellung erforderlich, solange die benötigten Szeneninformationen erhalten bleiben.

Deshalb erfolgt die Vorhersage in einer Bird's Eye View (BEV)-Darstellung. Die eingefrorenen BEVFusion-Konfigurationen [3] liefern für Segmentierung und Detektion jeweils Repräsentationen in einem eigenen BEV-Latentraum. Die einzelnen Zustände in diesem Merkmalsraum werden im Folgenden als latente BEV-Repräsentationen, kurz Latents, bezeichnet. Trainiert wird nur das Prädiktionsmodell. Kleine Prädiktoren auf eingefrorenen Merkmalen wurden bereits in aktuellen Arbeiten untersucht [4], [5]. Offen bleibt, wie gut sich dieser Ansatz auf die beiden BEV-Latenträume übertragen lässt und wie die vorhergesagten Repräsentationen mit den ursprünglichen Wahrnehmungsköpfen (engl. perception heads) bewertet werden können.

Als Prädiktor dient ein Transformer mit rund sechs Millionen Parametern, der aus drei aufeinanderfolgenden BEV-Repräsentationen die nächste Repräsentation vorhersagt. Die Eingaben stammen aus den jeweiligen auf nuScenes [6] trainierten BEVFusion-Konfigurationen und werden zur Bewertung mit den zugehörigen Segmentierungsbeziehungswise Detektionsköpfen verarbeitet.

Eine Persistenz-Baseline ohne gelerntes Dynamikmodell dient als einfache Referenz. Sie übernimmt den zuletzt beobachteten Zustand als Vorhersage. Für die Segmentierung wird zusätzlich eine ego-kompensierte Variante betrachtet, bei der die dekodierte Maske anhand der bekannten Eigenbewegung transformiert wird.

Ein Schwerpunkt der Arbeit liegt auf einem ressourceneffizienten Prädiktor. Neben der Modellgröße werden deshalb Laufzeit und Speicherbedarf betrachtet. Weitere Untersuchungen betreffen die Zielfunktion, die Bewertung der Vorhersagen und den verbleibenden Metrikabstand.

1.2 Forschungsfragen

Aus diesen Überlegungen ergeben sich mehrere offene Punkte. Zunächst ist zu prüfen, ob die gelernte Vorhersage gegenüber den verwendeten Persistenz-Baselines einen Vorteil bietet. Darüber hinaus ist offen, welche Bestandteile der Zielfunktion für Segmentierung und Detektion erforderlich sind und wie groß der verbleibende Abstand zur Auswertung mit realer Zielrepräsentation ist. Außerdem ist zu klären, ob sich dieser Abstand eher durch Änderungen am Weltmodell oder durch eine Anpassung der Wahrnehmungsköpfe verringern lässt. Daraus ergeben sich vier Forschungsfragen:

- Frage 1.** Übertrifft das Weltmodell die verwendeten Persistenz-Baselines?
- Frage 2.** Welche Terme der Trainings-Loss-Funktion verbessern die Vorhersage, und unterscheidet sich ihr Einfluss zwischen Segmentierung und Detektion?
- Frage 3.** Wie groß ist der Abstand des Weltmodells zur Referenz mit realer Zielrepräsentation, und wie groß ist der Abstand dieser Referenz zur perfekten Metrik?
- Frage 4.** Lässt sich die Leistung auf vorhergesagten BEV-Repräsentationen stärker durch Änderungen am Weltmodell oder durch eine Anpassung der nachgelagerten Köpfe verbessern?

1.3 Aufbau der Arbeit

Nach den Grundlagen und dem Stand der Technik in Kapitel 2 beschreibt Kapitel 3 das Konzept und Messverfahren. Die Umsetzung folgt in Kapitel 4, bevor Kapitel 5 die Ergebnisse auswertet. Abschließend fasst Kapitel 6 die wichtigsten Ergebnisse zusammen.

2 Grundlagen und verwandte Arbeiten

Dieses Kapitel führt zunächst die für die Arbeit benötigten technischen Grundlagen ein und ordnet anschließend verwandte Ansätze ein. Berücksichtigt werden Arbeiten zu Transformer-Modellen, generativer Vorhersage, Bird's Eye View (BEV)-basierter Wahrnehmung und Feature-Forecasting. Der Schwerpunkt liegt auf Verfahren, die zukünftige latente Merkmale vorhersagen oder eingefrorene Repräsentationen für nachgelagerte Aufgaben verwenden.

2.1 Transformer

Transformer verarbeiten eine Folge von Token mit Selbstaufmerksamkeit (Self-Attention) [7]. Ein Token ist dabei ein Vektor mit fester Dimension. Bei Bild- oder BEV-Daten kann ein Token einen räumlichen Ausschnitt der Eingabe beschreiben.

Für jeden Token werden drei Vektoren berechnet: Query, Key und Value. Query und Key bestimmen, wie stark zwei Token miteinander verknüpft werden. Die Values enthalten die Informationen, die anschließend gewichtet zusammengeführt werden. Für die gestapelten Matrizen $\mathbf{Q}$, $\mathbf{K}$ und $\mathbf{V}$ gilt

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (2.1)$$

wobei d_k die Dimension der Key-Vektoren ist [7]. Das Produkt $\mathbf{Q}\mathbf{K}^\top$ beschreibt die Ähnlichkeit zwischen den Token. Die Softmax-Funktion wandelt diese Werte in Gewichte um. Jeder Token erhält dadurch Informationen von allen anderen Token.

Bei Multi-Head-Attention wird dieser Vorgang mit unterschiedlichen Projektionen mehrfach parallel ausgeführt. Die Ergebnisse der einzelnen Aufmerksamkeitsköpfe werden zusammengeführt, sodass das Modell verschiedene Beziehungen zwischen den Token gleichzeitig abbilden kann.

Auf die Selbstaufmerksamkeit folgt ein Multilayer Perceptron (MLP), das als Feed-Forward-Netz jeden Token einzeln verarbeitet. Beide Schritte verwenden Residualverbindungen, bei denen die jeweilige Eingabe zur Ausgabe addiert wird (Abbildung 2.1). Dadurch muss eine Schicht nur die Änderung der bestehenden Repräsentation lernen.

Da die Selbstaufmerksamkeit weder die Reihenfolge noch die räumliche Position der Token berücksichtigt, wird zusätzliche Positionsinformation benötigt [7]. Beim Vision Transformer werden Bilder dazu in Patches zerlegt und als Token-Folge verarbeitet [8]. Dasselbe Prinzip lässt sich auf eine räumlich aufgeteilte Repräsentation im BEV-Latentraum übertragen.

Auch die Position der Layer Normalization (LN) beeinflusst das Training. Bei Post-LN liegt sie nach der Residualaddition, bei Pre-LN dagegen vor Self-Attention beziehungsweise Feed-Forward-Netz im Residualzweig. Da Pre-LN bei tieferen Transformern

die Trainingsstabilität verbessert [9], wird diese Anordnung im Prädiktionsmodell verwendet.

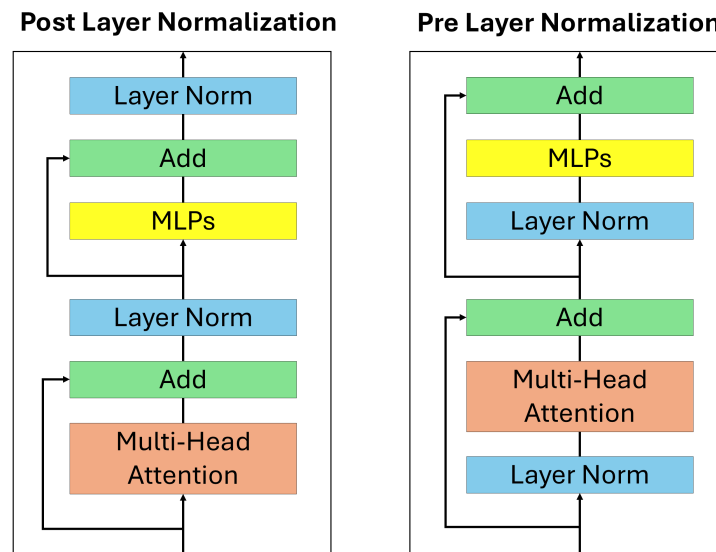


Abbildung 2.1: Transformer-Block in Post-LN- (links) und Pre-LN-Anordnung (rechts). Nach [7], [9].

2.2 Generative Modellierung

Bei Vorhersageaufgaben sind für denselben Ausgangszustand mehrere zukünftige Zustände möglich. Während ein deterministisches Modell eine einzelne Vorhersage erzeugt, können generative Modelle unterschiedliche Ausgaben erzeugen. In dieser Arbeit werden dafür Conditional Variational Autoencoder (CVAE), Diffusion und Flow Matching betrachtet.

Ein CVAE ergänzt das Modell um eine latente Zufallsvariable z [10], [11]. Im Training werden eine aus der Eingabe geschätzte Prior-Verteilung und eine zusätzlich vom Ziel abhängige Verteilung aneinander angenähert. Aus der Eingabe und einer gezogenen Stichprobe z erzeugt der Decoder anschließend die Ausgabe. Nutzt der Decoder z kaum, verhält sich das Modell nahezu deterministisch [12].

Diffusionsmodelle erzeugen Stichproben, indem Rauschen schrittweise entfernt wird [13]. Dafür muss das Modell mehrfach ausgewertet werden. Das generative Fahrmodell GAIA-2 verwendet dieses Verfahren in einem komprimierten Latentraum [2].

Beim Flow Matching wird dagegen ein Vektorfeld gelernt, das Stichproben aus einer einfachen Ausgangsverteilung in die Zielverteilung überführt [14]. Die Erzeugung kann mit weniger Integrationsschritten auskommen als bei klassischen Diffusionsverfahren. Als generative Alternative zur deterministischen Regression wurde Flow Matching bereits für die Merkmalsvorhersage eingesetzt [15].

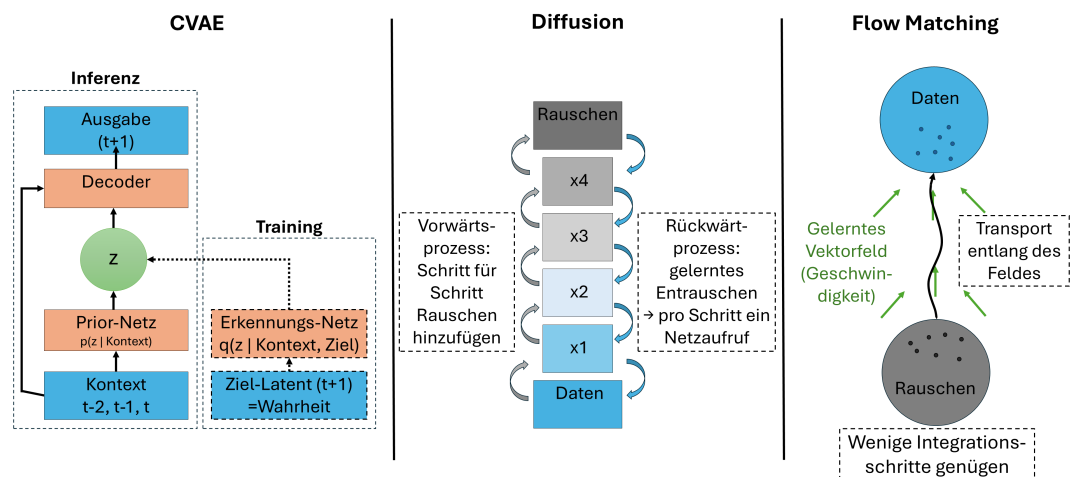


Abbildung 2.2: Drei Familien generativer Modelle im Vergleich: (a) CVAE mit latenter Variable, (b) Diffusion mit iterativem Entrauschen und (c) Flow Matching mit gelerntem Vektorfeld. Nach [13], [14], [16].

2.3 BEVFusion

Die BEV-Darstellung beschreibt die Fahrzeugumgebung als metrisches 2D-Gitter. Dadurch können Informationen verschiedener Sensoren in einem gemeinsamen Koordinatensystem verarbeitet werden.

Kameramerkmale müssen aus der Bildansicht in den BEV-Raum überführt werden. Lift-Splat-Shoot kombiniert dazu Bildmerkmale mit einer gelernten Tiefenverteilung und projiziert sie in das BEV-Gitter [17]. Im Gegensatz zu Kamerabildern liefern LiDAR-Sensoren (Light Detection And Ranging) direkt 3D-Punkte im Sensorkoordinatensystem. Diese können ohne zusätzliche Tiefenschätzung in eine räumliche Darstellung überführt werden.

BEVFusion kombiniert Kamera- und LiDAR-Merkmale zu einer gemeinsamen Repräsentation im BEV-Latentraum [3]. Darauf arbeiten getrennte Wahrnehmungsköpfe für semantische Kartensegmentierung und 3D-Objektdetektion. Als Datensatz verwendet BEVFusion unter anderem nuScenes. Neben Kamerabildern enthält nuScenes auch Messungen aus Radio Detection And Ranging (Radar) und LiDAR [6].

Für die weitere Arbeit sind insbesondere die fusionierte BEV-Repräsentation und die zugehörigen Wahrnehmungsköpfe relevant. Diese Schnittstelle ermöglicht später die Auswertung vorhergesagter Repräsentationen mit den ursprünglichen Segmentierungs- und Detektionsköpfen.

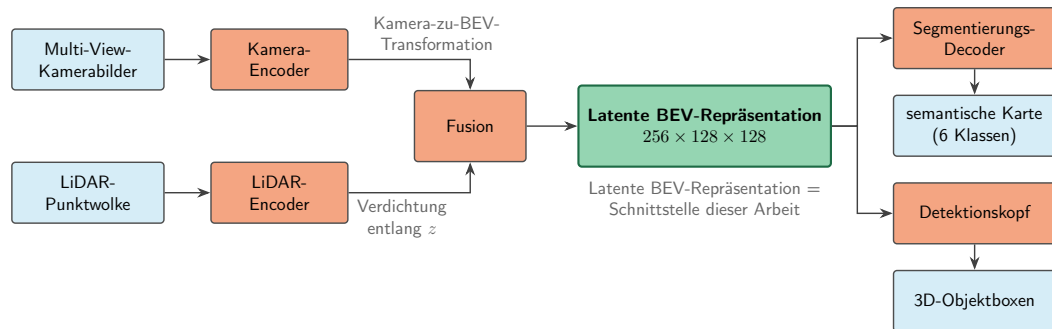


Abbildung 2.3: BEV-Wahrnehmungspipeline: Kamera- und LiDAR-Merkmale werden in ein gemeinsames BEV-Gitter überführt und fusioniert. Die hervorgehobene BEV-Repräsentation bildet die Schnittstelle zu den Segmentierungs- und Detektionsköpfen. Nach [3].

2.4 Weltmodelle und Feature-Forecasting

Weltmodelle sagen zukünftige Zustände einer Umgebung voraus. Für das automatisierte Fahren unterscheiden sich bestehende Ansätze vor allem darin, welche Repräsentation vorhergesagt wird [1], [18]. Dazu gehören Videobilder, Belegungsgitter und latente Merkmale.

Bildbasierte Modelle erzeugen zukünftige Kamerabilder oder Videos. Beispiele sind GAIA-2 [2] und OmniDreams [19]. Andere Arbeiten verwenden strukturierte Zustände, etwa zukünftige Belegungsgitter oder deren zeitliche Änderung [20], [21].

Eine weitere Gruppe nutzt latente Zustände direkt für die Planung. LAW verwendet die Vorhersage zukünftiger latenter Repräsentationen als Hilfsziel für einen durchgängig trainierten Fahransatz [22]. Andere Ansätze bewerten Trajektorien mit einem Weltmodell im BEV-Latentraum [23], modellieren Fahrintentionen im Latentraum [24] oder konditionieren die Planung auf vorhergesagte zukünftige latente Repräsentationen [25].

Für diese Arbeit sind vor allem Modelle relevant, die zukünftige Merkmale eines Wahrnehmungsmodells direkt vorhersagen.

Feature-Forecasting. Beim Feature-Forecasting wird aus vergangenen Merkmalen eines Wahrnehmungs- oder Grundlagenmodells ein zukünftiges Merkmal vorhergesagt.

LAW sagt die nächste latente Repräsentation des eigenen Fahrmodells vorher und verwendet sie als selbstüberwachtes Hilfsziel für das durchgängige Training des Fahrmodells [22]. Eine eigenständige Bewertung dieser Vorhersage als Wahrnehmungsausgabe erfolgt dort nicht.

DINO-Foresight behandelt Feature-Forecasting dagegen als eigenständige Aufgabe [26]. Ein Transformer sagt Merkmale eines eingefrorenen Vision-Modells vorher, die anschließend mit dedizierten Wahrnehmungsköpfen für Segmentierung, Tiefe und Oberflächennormalen ausgewertet werden.

LeWorldModel verwendet einen kompakten Prädiktor mit rund 15 Millionen Parametern [5]. Die Zielfunktion besteht aus einem Vorhersagefehler und einem Regularisierungsterm für die Statistik der Embeddings. Encoder und Prädiktor werden dabei gemeinsam trainiert. Als Zustand verwendet das Modell einen globalen Vektor ohne Decoder zurück in eine Wahrnehmungsausgabe.

Weitere Arbeiten untersuchen unter anderem Planung direkt auf latenten Merkmalen [27], Delta-Token zur Reduktion der Modellgröße [28], Masked Modeling für semantische Zukunftsvorhersage [29] und BEV-Feature-Forecasting als Vortrainingsziel [30].

Deterministische und generative Vorhersage. Feature-Forecasting kann deterministisch oder generativ formuliert werden. DINO-Foresight verwendet eine deterministische Smooth-L1-Regression [26], während VFMM auf einem eingefrorenen Merkmalsraum autoregressives Flow Matching einsetzt [15]. LeWorldModel ergänzt die deterministische Vorhersage dagegen um einen Regularisierungsterm für die Verteilung der Embeddings [5]. Daraus ergeben sich für diese Arbeit drei Varianten: deterministische Regression, Regression mit zusätzlichen Statistiktermen und generative Vorhersage.

Eingefrorene und mittrainierte Encoder. Auch beim Training des Encoders unterscheiden sich die Ansätze. Mehrere Arbeiten verwenden eingefrorene Encoder [4], [31], [32]. In einem kontrollierten Vergleich übertraf ein eingefrorener Videoencoder mit kleinem Prädiktor die mittrainierte Variante [4].

Andere Verfahren trainieren Encoder und Prädiktor gemeinsam. Dazu gehören Ansätze auf Basis einer Joint-Embedding Predictive Architecture (JEPA) für Video- und LiDAR-Daten [5], [33], [34]. Diese Arbeiten bewerten ihre Repräsentationen jedoch über die Planungsgüte und nicht mit den ursprünglichen Wahrnehmungsköpfen. TC-WM verwendet dagegen eingefrorene Grundlagenmerkmale und lernt darauf einen zusätzlichen kompakten Latentraum [35].

Einordnung der eigenen Arbeit. Die vorliegende Arbeit verwendet eingefrorene BEVFusion-Konfigurationen und sagt die Repräsentationen in den BEV-Latraäumen der jeweiligen Wahrnehmungsaufgabe direkt vorher. Zur Bewertung bleiben die zugehörigen Segmentierungs- und Detektionsköpfe erhalten.

Unter den betrachteten Arbeiten findet sich diese Kombination nicht. LAW sagt ebenfalls latente Repräsentationen des eigenen Fahrmodells vorher, verwendet sie jedoch nur als Hilfsziel für das durchgängige Training des Fahrmodells [22]. DINO-Foresight und VFMF arbeiten auf Merkmalen eines generischen Vision-Modells [15], [26]. LeWorldModel besitzt keinen Decoder zurück in eine Wahrnehmungsausgabe [5], während TC-WM einen zusätzlichen kompakten Latentraum lernt [35].

Dadurch können die Vorhersagen dieser Arbeit mit den vorhandenen Wahrnehmungsköpfen ausgewertet und direkt mit den Referenzannotationen verglichen werden. Das folgende Kapitel legt das dafür verwendete Messverfahren fest.

3 Konzept

Dieses Kapitel legt die verwendeten Bird's Eye View (BEV)-Latenträume, den Umgang mit dem Wahrnehmungsmodell, die Form der Vorhersage und das Messkonzept fest. Bildsynthese, Belegungsgitter und eine direkte Kopplung an die Planung werden nicht betrachtet, da die Forschungsfragen ausschließlich die Vorhersage und Bewertung von BEV-Repräsentationen auf den beiden Wahrnehmungsaufgaben betreffen. Der Zustand wird deshalb im jeweiligen BEV-Latentraum vorhergesagt und mit den Metriken der zugehörigen Wahrnehmungsaufgabe bewertet.

3.1 Anforderungen und Bewertungskriterien

Aus der Aufgabenstellung und den Forschungsfragen ergeben sich Anforderungen an den Prädiktor sowie Kriterien für die spätere Bewertung. Die folgenden sieben Punkte legen diese Randbedingungen fest. Kapitel 4 beschreibt ihre technische Umsetzung, Kapitel 5 die Auswertung.

- R1 – Ein-Schritt-Vorhersage** Das Modell sagt aus vergangenen BEV-Repräsentationen die unmittelbar folgende Repräsentation vorher. Mehrschrittige Vorhersagen entstehen autoregressiv.
- R2 – Persistenz-Referenz** Als Referenz ohne gelerntes Vorhersagemodell dient eine Persistenz-Baseline. In der Segmentierung wird zusätzlich eine ego-kompensierte Variante verwendet, bei der die dekodierte Maske anhand der bekannten Eigenbewegung transformiert wird. Das Weltmodell wird auf derselben Zielmetrik mit dieser Referenz verglichen.
- R3 – Auswertung durch Wahrnehmungsköpfe** Vorhergesagte BEV-Repräsentationen müssen mit den zugehörigen Wahrnehmungsköpfen auswertbar sein. Dazu werden sie anstelle der realen Zielrepräsentation in den jeweiligen BEVFusion-Pfad injiziert und anschließend gegen die Referenzannotation bewertet.
- R4 – Empirische Vergleichsschwelle** Wiederholte Trainingsläufe mit verschiedenen pseudozufälligen Startwerten (engl. random seeds) bestimmen die Streuung zwischen den Läufen. Unterschiede unterhalb dieser Streuung werden als Seed-Variabilität behandelt.
- R5 – Eingefrorener Referenzpfad** Encoder und Wahrnehmungsköpfe des ursprünglichen Wahrnehmungsmodells bleiben im Referenzpfad unverändert. Gezielte Kopfadaptationen werden ausschließlich an separaten Kopien durchgeführt.
- R6 – Ressourceneffizienz** Der Prädiktor bleibt in der Größenordnung weniger Millionen Parameter und ist auf einer einzelnen Grafikkarte trainierbar.
- R7 – Modulare Zielfunktion** Jeder Loss-Term besitzt ein eigenes Gewicht und kann separat aktiviert oder deaktiviert werden. Dadurch lassen sich die Beiträge der einzelnen Terme in der Loss-Ablation untersuchen.

Durch die Verwendung der nuScenes-Schlüsselbilder (engl. keyframes) beträgt die Schrittweite zwischen zwei Frames eine halbe Sekunde [6]. Ein Vorhersageschritt entspricht damit demselben Zeithorizont. Als Kontext dienen zunächst drei vergangene Frames, während längere Kontexte separat in der Evaluierung untersucht werden. Die bekannte Eigenbewegung und das eingefrorene Wahrnehmungsmodell gehören zu den festen Messbedingungen.

3.2 Entwurfsentscheidungen

Aufgabenspezifische BEV-Latenträume. Vorhergesagt wird die fusionierte BEV-Repräsentation von BEVFusion (Abschnitt 2.3). Für Segmentierung und Detektion stammen die Repräsentationen aus den jeweils zugehörigen BEVFusion-Konfigurationen. Die Prädiktoren werden für beide Latenträume getrennt trainiert und ausgewertet. Da die passenden Wahrnehmungsköpfe bereits vorhanden sind, können die Vorhersagen direkt mit den ursprünglichen Wahrnehmungsmetriken bewertet werden.

Dazu wird die vorhergesagte Repräsentation anstelle der realen Zielrepräsentation in den jeweiligen Kopf eingespeist und dessen Ausgabe gegen die Referenzannotation bewertet. Dieses Verfahren wird im Folgenden als Injektion bezeichnet. Da auch der eingefrorene Wahrnehmungspfad die erreichbare Zielmetrik begrenzt, wird zusätzlich die Referenz mit der realen Zielrepräsentation ausgewertet.

Eingefrorenes Wahrnehmungsmodell. Im Referenzpfad bleiben Encoder und Wahrnehmungsköpfe unverändert. Dadurch durchlaufen Persistenz, Weltmodell und die Referenz mit realer Zielrepräsentation dieselben nachgelagerten Komponenten. Trainiert wird zunächst ausschließlich der Prädiktor. Anpassungen der Köpfe werden später getrennt auf Kopien untersucht.

Architektur des Prädiktors. Der Prädiktor ist ein kleiner Transformer in Pre-LN-Anordnung (Abschnitt 2.1). Die Selbstaufmerksamkeit kann Informationen zwischen räumlich entfernten Bereichen und mehreren Zeitpunkten verknüpfen. Dabei soll die Modellgröße im Bereich weniger Millionen Parameter bleiben.

Da die Kosten der Selbstaufmerksamkeit quadratisch mit der Tokenzahl wachsen, wird die BEV-Repräsentation vor dem Transformer verdichtet und anschließend wieder auf die ursprüngliche Auflösung gebracht (Kapitel 4). Zusätzlich mischt ein gelerntes zellweises Gate die jüngste Kontextrepräsentation mit der Transformer-Vorhersage. Die Ausgabe kann dadurch lokal zwischen beiden Repräsentationen interpolieren. Ähnliche Residual-Ansätze werden auch in verwandten Weltmodellen verwendet [21]. Die Initialisierung nahe der Identitätsabbildung folgt [5].

Der Prädiktor ist in seiner Grundkonfiguration deterministisch und kann auf demselben Modellrumpf um CVAE- oder Flow-Matching-Köpfe erweitert werden (Abschnitt 2.2).

Zielfunktion als Baukasten. Die Zielfunktion ist modular aufgebaut und umfasst sechs Terme: einen Rekonstruktionsfehler auf mehreren räumlichen Auflösungen sowie Terme für Merkmalsrichtung, Kanalstatistik und räumliche Struktur. Jeder Term besitzt ein eigenes Gewicht und kann einzeln deaktiviert werden (R7). Die Formeln werden in Kapitel 4 angegeben. In der anschließenden Ablation wird geprüft, welche Terme die Zielmetrik messbar verändern.

3.3 Messkette und Untersuchungsplan

Da für die Ablation viele Konfigurationen verglichen werden, ist zunächst eine schnelle Vorauswahl erforderlich. Die endgültige Bewertung erfolgt anschließend auf der vollständigen Validierungsmenge mit den Wahrnehmungsmetriken. Daraus ergibt sich eine dreistufige Messkette.

In der ersten Stufe wird auf einem festen Ausschnitt der Validierungsdaten die dekodierte Vorhersage mit der dekodierten Ausgabe der realen BEV-Repräsentation verglichen. Anschließend werden ausgewählte Konfigurationen nach beendetem Training auf der vollständigen Validierungsmenge ausgewertet. Den Abschluss bildet die Injektion gegen die Referenzannotation, mit der die Vorhersage direkt auf der jeweiligen Wahrnehmungsaufgabe gemessen wird.

Zur Einordnung der Ergebnisse dient zunächst die reale BEV-Repräsentation des Zielzeitpunkts, die mit demselben Wahrnehmungskopf ausgewertet wird. Gemessen werden sowohl ihr Abstand zur perfekten Metrik als auch der Abstand des Weltmodells zu dieser Referenz. Ergänzend dient die Persistenz als Referenz ohne gelerntes Vorhersagemodell.

Identische Konfigurationen werden mit mehreren pseudozufälligen Startwerten wiederholt. Die beobachtete Streuung dient als empirische Schwelle zur Einordnung kleiner Unterschiede. Werte darunter werden als Seed-Variabilität berichtet.

Abbildung 3.1 zeigt das Gesamtsystem. Der Prädiktor arbeitet an der Schnittstelle der BEV-Repräsentation von BEVFusion. Vorhersage, reale Zielrepräsentation und Persistenz werden durch die jeweils zugehörigen Wahrnehmungsköpfe ausgewertet.

Als Vergleich zum Weltmodell dient die Persistenz-Baseline. Die Loss-Ablation untersucht den Einfluss der Zielfunktion, während die Referenz mit realer Zielrepräsentation den verbleibenden Metrikabstand einordnet. Änderungen am Weltmodell und angepasste Kopien der nachgelagerten Köpfe werden anschließend miteinander verglichen. Kapitel 4 beschreibt die Umsetzung, Kapitel 5 die Auswertung.

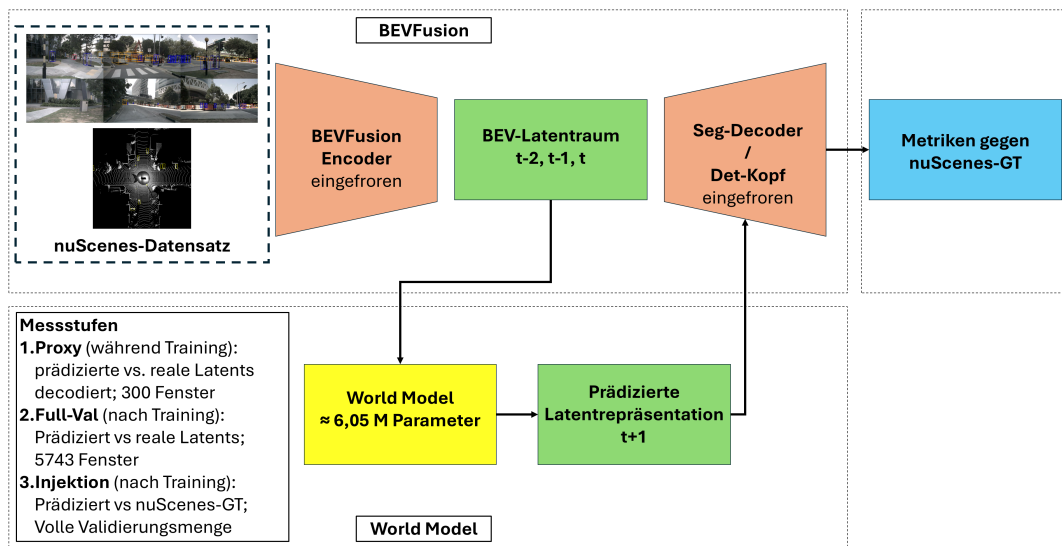


Abbildung 3.1: Gesamtsystem der Arbeit. Der eingefrorene BEVFusion-Encoder liefert BEV-Repräsentationen. Das Weltmodell sagt aus drei Kontextframes die Repräsentation des nächsten Zeitpunkts vorher. Die Vorhersage wird per Injektion durch die unveränderten Köpfe gegen die nuScenes-Annotation bewertet. Als Bezugspunkte dienen die Referenz mit realer Zielrepräsentation ($t + 1$ durch denselben Kopf) und die Persistenz-Baseline. Wahrnehmungspipeline nach [3].

4 Implementierung

4.1 Systemüberblick und Datenpipeline

Abbildung 4.1 zeigt den Prädiktor im Datenfluss. Die einzelnen Module werden in Abschnitt 4.2 beschrieben. Für Segmentierung und Detektion existieren getrennte Stränge. Der Segmentierungsstrang verwendet die Latents der Segmentierungskonfiguration, während der Detektionsstrang auf den größeren Latents der Detektionskonfiguration arbeitet.

Softwareseitig ist das System in zwei Pipelines getrennt.¹ Die Extraktion der Latents und ihre Injektion in die Wahrnehmungsköpfe laufen im BEVFusion-Docker-Container, sodass Encoder und Köpfe in ihrer Originalumgebung ausgeführt werden. Training, Zielfunktion und Auswertung des Prädiktors erfolgen dagegen in einer eigenständigen PyTorch-Pipeline ohne BEVFusion-Abhängigkeit. Trainiert wurde auf einer Workstation mit einem einzelnen Grafikprozessor (GPU) und einem DGX-Cluster mit vier Knoten zu je acht A100-Grafikkarten. Pro Trainingslauf wird nur eine GPU verwendet. Parallelisiert werden auf dem Cluster ausschließlich unabhängige Versuche.

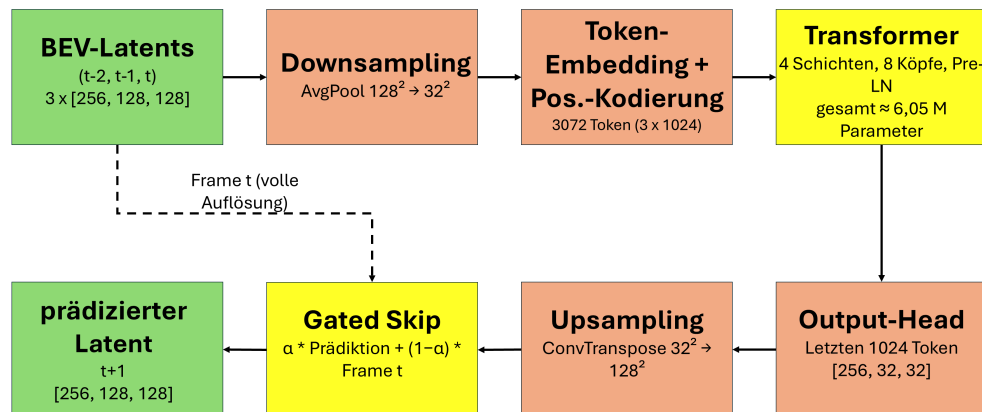


Abbildung 4.1: Aufbau des Prädiktors: Drei BEV-Latents werden per Average-Pooling verdichtet, als 3072 Token durch den vierschichtigen Pre-LN-Transformer geführt, per transponierter Faltung auf die volle Auflösung angehoben und über das zellweise gelernte Gate mit dem letzten Kontext-Latent gemischt ($\alpha \in [0, 1]$ je Zelle: 0 = kopieren, 1 = vorhersagen).

Datenpipeline. Zur Extraktion greift ein Hook im Vorwärtspfad von BEVFusion das fusionierte BEV-Latent nach der Sensorfusion und vor den Wahrnehmungsköpfen

¹Der Quellcode dieser Arbeit ist verfügbar unter <https://git.unibw.de/j22m0657/masterarbeit-bev-wm>.

ab. Dabei läuft das Wahrnehmungsmodell im Inferenzmodus und seine Gewichte bleiben unverändert. Im Segmentierungsstrang liefert ein Extraktionslauf 28 130 Trainings- und 6 019 Validierungs-Latents der Form $256 \times 128 \times 128$, zusammen etwa 535 GB. Für den Detektionsstrang wird derselbe Mechanismus verwendet, dort haben die Latents die Form $256 \times 180 \times 180$.

Aus vier aufeinanderfolgenden Frames bildet der Datenlader jeweils ein Fenster mit drei Kontextframes und einem Vorhersageziel. Szenengrenzen werden dabei nicht überschritten. Auf der Validierungsmenge entstehen 5 743 Fenster, die allen späteren Vollausswertungen zugrunde liegen.

Beim Training erwies sich der Datenpfad als begrenzender Faktor. Deshalb werden alle Latents eines Splits in einer Memory-Mapped-Datei zusammengefasst und beim Jobstart in den lokalen Arbeitsspeicher kopiert. Zur Reduktion des Speicherbedarfs werden die Daten von float32 nach float16 konvertiert. Beim Erzeugen wird die gepackte Datei automatisch mit den Einzeldateien verglichen. Da der Formatwechsel vor Beginn der Loss-Versuchsreihen erfolgte, verwenden alle Varianten innerhalb einer Versuchsreihe identische Eingabedaten.

4.2 Prädiktionsmodell

Der Prädiktor setzt die Entwurfsentscheidungen aus Abschnitt 3.2 in fünf Module um. Zunächst verdichtet ein Average-Pooling mit Kernbreite und Schrittweite 4 jede Kontextkarte von 128×128 auf 32×32 Zellen. Aus jeder Zelle entsteht ein Token der Dimension 256, sodass ein Vorwärtsdurchlauf $3 \times 1024 = 3072$ Token verarbeitet. Positionsinformationen werden getrennt für beide Raumrichtungen und die Zeitachse addiert. Anschließend verarbeitet ein Transformer mit vier Pre-LN-Schichten, jeweils acht Köpfen und einer Feed-Forward-Dimension von 1024 die Token. Für die Ausgabe werden die 1024 Token des jüngsten Kontextframes verwendet und über eine Linearschicht mit Gaussian Error Linear Unit (GELU)-Aktivierung in eine Karte der Form $256 \times 32 \times 32$ zurückgeführt. Zwei transponierte Faltungen mit Gruppennormalisierung und GELU erhöhen die Auflösung auf 128×128 , bevor eine abschließende Faltung das vorhergesagte Latent erzeugt.

Den Abschluss bildet ein zellweises Gate. Aus dem Kandidaten $\hat{\mathbf{x}}_{\text{trans}}$ und dem letzten Kontext-Latent $\mathbf{x}_t$ erzeugt eine 1×1 -Faltung mit Sigmoid-Aktivierung die Gewichtskarte $\alpha \in [0, 1]^{128 \times 128}$. Die endgültige Vorhersage ergibt sich als zellweise Mischung

$$\hat{\mathbf{x}}_{t+1} = \alpha \odot \hat{\mathbf{x}}_{\text{trans}} + (1 - \alpha) \odot \mathbf{x}_t, \quad (4.1)$$

Dabei bezeichnet $\odot$ die elementweise Multiplikation. Für α nahe null dominiert das letzte Kontext-Latent, für α nahe eins übernimmt die Transformer-Vorhersage. Die Verteilung der Gate-Werte wird in Kapitel 5 untersucht.

Der Prädiktor umfasst rund 6,05 Millionen Parameter. Im Detektionsstrang wird dieselbe Architektur mit einer Gittergröße von 45×45 Zellen verwendet, wodurch

pro Durchlauf 6075 Token entstehen. Tabelle 4.1 fasst die Architekturparameter beider Stränge zusammen.

Tabelle 4.1: *Architekturparameter des Prädiktors.*

Parameter	Wert
Kontextframes	3
Token-Dimension d_{model}	256
Transformer-Schichten (Pre-LN)	4
Aufmerksamkeitsköpfe	8
Feed-Forward-Dimension	1024
Gitter nach Verdichtung (Segmentierung / Detektion)	32×32 / 45×45
Token je Vorwärtsdurchlauf (Segmentierung / Detektion)	3072 / 6075
Parameter gesamt	$\approx 6,05$ M

4.3 Zielfunktion und Training

Ausgehend von Abschnitt 3.2 wird die Zielfunktion aus sechs gewichteten Termen aufgebaut. Für die Vorhersage $\hat{\mathbf{x}}$ aus Gleichung (4.1) und das Ziel-Latent $\mathbf{x}$, beide mit der Form $C \times H \times W$ und $C = 256$, ergibt sich:

$$\mathcal{L} = \lambda_{\text{rec}} \mathcal{L}_{\text{rec}} + \lambda_{\text{cos}} \mathcal{L}_{\text{cos}} + \lambda_{\text{mean}} \mathcal{L}_{\text{mean}} + \lambda_{\text{std}} \mathcal{L}_{\text{std}} + \lambda_{\text{grad}} \mathcal{L}_{\text{grad}} + \lambda_{\text{ssim}} \mathcal{L}_{\text{ssim}}. \quad (4.2)$$

Jedes Gewicht λ_i ist ein Konfigurationsparameter. Mit $\lambda_i=0$ wird der zugehörige Term vollständig deaktiviert.

Der Rekonstruktionsterm misst den punkweisen Fehler auf drei räumlichen Auflösungen:

$$\mathcal{L}_{\text{rec}} = \frac{1}{3} [d(\hat{\mathbf{x}}, \mathbf{x}) + d(\hat{\mathbf{x}}_{\downarrow 2}, \mathbf{x}_{\downarrow 2}) + d(\hat{\mathbf{x}}_{\downarrow 4}, \mathbf{x}_{\downarrow 4})], \quad (4.3)$$

Dabei bezeichnet $\downarrow k$ ein Average-Pooling um den Faktor k . Die verdichteten Ebenen erfassen gröbere räumliche Strukturen, während die volle Auflösung lokale Details berücksichtigt. Als punkweise Distanz d kann entweder der quadratische Fehler oder Smooth-L1 verwendet werden.

$$d(e) = \begin{cases} \frac{e^2}{2\beta} & \text{für } |e| < \beta, \\ |e| - \frac{\beta}{2} & \text{sonst,} \end{cases} \quad \beta = 1, \quad (4.4)$$

Bei kleinen Residuen verhält sich Smooth-L1 quadratisch, bei großen Residuen linear. Große Fehler erhalten dadurch weniger Gewicht als bei einem rein quadratischen Fehler. Auch DINO-Foresight verwendet Smooth-L1 für die Merkmalsvorhersage [26].

Der Kosinus-Term vergleicht dagegen die Richtung der Merkmalsvektoren unabhängig von ihrer Amplitude:

$$\mathcal{L}_{\text{cos}} = 1 - \frac{\langle \text{vec}(\hat{\mathbf{x}}), \text{vec}(\mathbf{x}) \rangle}{\|\text{vec}(\hat{\mathbf{x}})\|_2 \|\text{vec}(\mathbf{x})\|_2}. \quad (4.5)$$

Mittelwert μ_c und Standardabweichung σ_c jedes Kanals werden zusätzlich an die Statistik des Ziel-Latents gebunden:

$$\mathcal{L}_{\text{mean}} = \frac{1}{C} \sum_{c=1}^C (\mu_c(\hat{\mathbf{x}}) - \mu_c(\mathbf{x}))^2, \quad \mathcal{L}_{\text{std}} = \frac{1}{C} \sum_{c=1}^C (\sigma_c(\hat{\mathbf{x}}) - \sigma_c(\mathbf{x}))^2. \quad (4.6)$$

Räumliche Übergänge erfasst der Gradiententerm, der finite Differenzen entlang beider Achsen über den Mean Squared Error (MSE) vergleicht:

$$\mathcal{L}_{\text{grad}} = \text{MSE}(\Delta_x \hat{\mathbf{x}}, \Delta_x \mathbf{x}) + \text{MSE}(\Delta_y \hat{\mathbf{x}}, \Delta_y \mathbf{x}). \quad (4.7)$$

Für die lokale räumliche Struktur wird zusätzlich Structural Similarity Index (SSIM) verwendet:

$$\mathcal{L}_{\text{ssim}} = 1 - \text{SSIM}(\hat{\mathbf{x}}, \mathbf{x}) = 1 - \frac{(2\mu_{\hat{x}}\mu_x + c_1)(2\sigma_{\hat{x}x} + c_2)}{(\mu_{\hat{x}}^2 + \mu_x^2 + c_1)(\sigma_{\hat{x}}^2 + \sigma_x^2 + c_2)}, \quad (4.8)$$

mit fensterweisen Mittelwerten, Varianzen und Kovarianz sowie den üblichen Stabilisierungskonstanten c_1, c_2 .

Als Ausgangskonfiguration erhält der Rekonstruktionsterm das Gewicht $\lambda_{\text{rec}} = 1,0$. Kosinus, Kanalmittel, Kanalstreuung und SSIM werden jeweils mit 0,1 gewichtet, während der Gradiententerm deaktiviert bleibt. Für die punktweise Distanz wird zunächst der quadratische Fehler verwendet. Diese Einstellung bildet den Ausgangspunkt der Loss-Studie in Kapitel 5.

Training. Optimiert wird mit AdamW [36] bei einer Lernrate von 10^{-4} , einem Weight Decay von 10^{-4} , Cosine-Annealing und gemischter Präzision. Ein Lauf umfasst maximal 50 Epochen bei Batchgröße 8. Als Abbruchkriterium dienen zehn Epochen ohne Verbesserung des Validierungs-Losses. Die Zufallsaat ist konfigurierbar, sodass identische Einstellungen mit mehreren Seeds wiederholt werden können.

Im Segmentierungsstrang wird der beste Checkpoint zusätzlich über die Zielmetrik bestimmt, da der Validierungs-Loss nur den Abstand zum Ziel-Latent erfasst. Dafür dekodiert der Trainingsloop alle fünf Epochen eine feste Teilstichprobe von 300 Validierungsfenstern mit dem eingefrorenen Segmentierungskopf und berechnet die mIoU (Abschnitt 4.4). Neben dem Checkpoint mit dem niedrigsten Loss wird so auch der Checkpoint mit der höchsten mIoU gespeichert. Für die spätere Auswertung wird letzterer verwendet.

Der Dekodierschritt läuft in 32-Bit-Gleitkommapräzision und entspricht damit der ursprünglichen Konfiguration des Segmentierungskopfs. Im Detektionsstrang steht dieser Nachbau in der Trainingsumgebung nicht zur Verfügung. Dort erfolgt die Checkpoint-Auswahl über den Validierungs-Loss, während die Metrik anschließend über die Injektionskette bestimmt wird.

Für vergleichbare Versuchsreihen werden alle Varianten eines Sweeps programmatisch aus derselben Basiskonfiguration erzeugt und ändern jeweils nur einen Parameter. Die besten Varianten einer Reihe werden anschließend in separaten Läufen ohne vorzeitiges Stoppen über 50 Epochen trainiert. Ihre endgültige Bewertung erfolgt auf allen 5 743 Validierungsfenstern, sodass die berichteten Werte aus Läufen mit gleicher Trainingsdauer stammen.

4.4 Messkette

Zur ersten Messstufe wird der Segmentierungskopf in der Trainingsumgebung benötigt. Da dieser nur im BEVFusion-Container verfügbar ist und dessen Abhängigkeiten nicht mit der Trainingsumgebung kompatibel sind, wurde die Decoder-Kette aus

Backbone, Neck, Gittertransformation und Klassifikator als PyTorch-Module nachgebaut. Der Backbone verarbeitet die BEV-Merkmale weiter, der Neck bereitet sie für die anschließende Gittertransformation und Klassifikation auf. Dabei werden exakt die Gewichte des Originals verwendet. Zur Prüfung wurden alle 98 Gewichtstensenoren im strikten Modus geladen und die dekodierte mIoU auf Referenz-Latents mit der Container-Referenz verglichen. Beide Pfade stimmen bis auf die dritte Nachkommastelle überein. Damit kann die feste Teilstichprobe mit 300 Fenstern während des Trainings ausgewertet werden. Zusätzlich wird die mittlere Kanalstreuung von Vorhersage und realem Latent verglichen. Ein Verhältnis von eins entspricht gleicher Streuung. In der zweiten Messstufe werden ausgewählte Modelle mit derselben Decoder-Kette auf allen 5 743 Validierungsfenstern ausgewertet.

Den Abschluss bildet die Bewertung gegen die Referenzannotation. Im Segmentierungsstrang wird das vorhergesagte Latent mit dem eingefrorenen Kopf zur Segmentierungskarte dekodiert. Für die Detektion wurde auf einen Nachbau des nachgelagerten Wahrnehmungspfads verzichtet. Stattdessen ersetzt ein Injektions-Hook im BEVFusion-Container das intern berechnete BEV-Latent durch die Vorhersage, während Detektionskopf und offizielle Bewertungswerkzeuge unverändert bleiben.

Mit einer rund 2,5-mal höheren Streuung liegen die Detektions-Latents in einem anderen Wertebereich als die Segmentierungs-Latents. Für das Training werden Eingabe und Ziel deshalb mit demselben Faktor skaliert, sodass die Smooth-L1-Distanz in beiden Strängen in einem vergleichbaren Wertebereich liegt. Vor der Injektion wird diese Skalierung wieder aufgehoben.

Zur Prüfung der Injektionskette wurden reale Latents eingespeist. Im Detektionsstrang werden damit die Referenzwerte des ursprünglichen Modells innerhalb der numerischen Genauigkeit reproduziert. Entsprechend wird im Segmentierungsstrang das reale Latent mit dem zugehörigen Kopf dekodiert (Abschnitt 3.3). Zusätzlich wurde geprüft, dass Skalierung, Speicherung in halber Genauigkeit und anschließende Rückskalierung die Metriken nicht messbar verändern.

Da für die ersten drei Frames einer Szene noch nicht genügend Kontext vorhanden ist, wird dort bei allen Varianten das reale Latent verwendet. Diese Behandlung ist für Modelle und Baselines identisch.

Als Baselines dienen eine naive und eine ego-kompensierte Persistenz. Die naive Variante übernimmt das letzte beobachtete Latent unverändert. Bei der ego-kompensierten Segmentierungsbaseline wird dagegen aus den Fahrzeugposen die relative Transformation in der Bodenebene bestimmt und auf die dekodierte Maske des letzten Kontextframes angewendet. Bei der Transformation werden außerhalb des Gitters liegende Werte verworfen; neu entstehende Randbereiche werden mit null belegt, was bei den binären Klassenmasken keiner Klasse entspricht. Geprüft wurde die Umsetzung sowohl mit einer Identitätstransformation als auch anhand der bekannten Fahrzeugbewegung.

In Kapitel 5 werden die naive und die ego-kompensierte Persistenz miteinander verglichen.

Kleine Unterschiede zwischen Modellvarianten werden anhand wiederholter Trainingsläufe eingeordnet. In der Segmentierung wurde dieselbe Konfiguration mit den Seeds 42, 43 und 44 trainiert. Aus deren Streuung ergibt sich eine empirische Schwelle von 0,014 mIoU.

Für die Detektion wurden zwei Referenzkonfigurationen mit jeweils zwei Zufallssaaten trainiert. Die maximale beobachtete Streuung beträgt 0,0044 mAP. Als konservative Vergleichsschwelle wird deshalb der doppelte Wert von 0,0088 mAP verwendet. Unterschiede darunter werden in der Evaluierung als Seed-Variabilität behandelt.

4.5 Erweiterungen und Versuchsdokumentation

Generative Vorhersageköpfe. Als generative Erweiterungen des deterministischen Prädiktors werden eine CVAE- und eine Flow-Matching-Variante untersucht (Abschnitt 2.2). Auf eine zusätzliche Diffusionsvariante wird wegen des höheren Aufwands für die Stichprobenerzeugung verzichtet. Flow Matching wurde bereits für die generative Merkmalsvorhersage eingesetzt [15].

Die CVAE-Variante verwendet ein Prior-Netz für die Verteilung der latenten Variable z . Während des Trainings kommt zusätzlich ein Erkennungsnetz zum Einsatz, das auch das Ziel-Latent erhält [37]. Über Feature-wise Linear Modulation (FiLM) wird die gezogene Stichprobe in die Token eingebracht [38]. Zur Regularisierung dient ein Kullback-Leibler-Term mit dem Gewicht β , das während der ersten fünf Epochen linear erhöht wird. Zusätzlich werden 0,05 nats Free Bits je Dimension verwendet [12], [39]. Ein Nat ist die Informationseinheit bei Verwendung des natürlichen Logarithmus. Bei Free Bits werden Beiträge des Kullback-Leibler-Terms unterhalb dieser Schwelle nicht weiter minimiert. Für die untersuchten β -Werte wird die Variante jeweils von Grund auf trainiert.

Beim Flow Matching dient die für die Segmentierung ausgewählte deterministische Konfiguration als feste Basis. Modelliert wird nur das Residuum

$$\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}}_{\text{det}}.$$

Das Geschwindigkeitsfeld besteht aus acht Residual-Faltungsblöcken und wird über FiLM mit dem Zeitschritt konditioniert. Trainiert wird mit der Flow-Matching-Loss-Funktion [14]:

$$\mathcal{L}_{\text{FM}} = \left\| \mathbf{v}_{\theta}(\mathbf{r}_{\tau}, \tau, \mathbf{c}) - (\mathbf{r} - \epsilon) \right\|^2. \quad (4.9)$$

Die Stichproben werden mit Euler-Integration erzeugt. Der Flow-Matching-Kopf umfasst rund 6,5 Millionen Parameter. Zusätzlich werden eine Skalierung des Residuums und eine Best-of- K -Auswertung untersucht. Bei Best-of- K wird anhand des bekannten Ziels die beste aus mehreren Stichproben ausgewählt. Die Auswertung dient daher ausschließlich der Analyse und ist nicht als einsetzbares Auswahlverfahren gedacht.

Adaptierte Wahrnehmungsköpfe. Zur Untersuchung der Kopfadaptation werden Kopien der nachgelagerten Wahrnehmungskomponenten auf vorhergesagten Latents nachtrainiert. Der Prädiktor und die ursprünglichen Köpfe bleiben dabei unverändert. Als Ziel dienen die nuScenes-Annotationen. Für die Auswertung wird anschließend nur der jeweilige Kopf-Checkpoint ausgetauscht.

Im Segmentierungsstrang werden Backbone, Neck und Segmentierungskopf mit insgesamt rund 9,3 Millionen Parametern nachtrainiert. Dafür stehen 27 038 Vorhersagefenster des für die Segmentierung ausgewählten Weltmodells mit den zugehörigen Referenzmasken zur Verfügung. Das Training umfasst acht Epochen mit der ursprünglichen Focal-Loss-Funktion.

Für die Detektion werden die nachgelagerten Schichten bis einschließlich des Detektionskopfes auf jedem zweiten Vorhersagefenster nachtrainiert. Dadurch stehen 13 519 Fenster mit Zielboxen aus den nuScenes-Annotationen zur Verfügung. Die erste Variante wird über sechs Epochen trainiert.

Eine zweite Variante erweitert das Training auf zwölf Epochen und verwendet einen Cosine-Lernratenverlauf. Der beste Checkpoint wird anhand des Validierungs-Losses auf 500 Vorhersagefenstern ausgewählt. Zusätzlich stammen bei 20 % der Trainingsfenster die Eingaben aus realen Latents. Diese Beimischung realer Latents soll die Leistung des angepassten Kopfes auf ihnen möglichst erhalten.

Versuchsdokumentation. Versionierte Skripte erzeugen die Abbildungen aus gespeicherten Auswertungsergebnissen. Die zentralen Ergebnisse liegen zusätzlich in Tabellenform vor. Zufallssaaten und Sweep-Konfigurationen sind in den jeweiligen Konfigurationsdateien dokumentiert. Die Varianten eines Sweeps werden aus einer gemeinsamen Basiskonfiguration erzeugt und unterscheiden sich jeweils nur in den gezielt veränderten Parametern.

5 Evaluierung der Vorhersage von BEV-Repräsentationen

5.1 Referenzwerte und Seed-Variabilität

Als Referenzen dienen die ego-kompensierte Persistenz und die reale Zielrepräsentation, die mit dem unveränderten Wahrnehmungskopf ausgewertet wird. Die zugehörigen Werte sind in Abbildung 5.1 und Tabelle 5.1 dargestellt.

Im Segmentierungsstrang erfolgt die Bewertung auf zwei Skalen. Auf der Proxy-Skala wird die dekodierte Vorhersage mit der dekodierten Ausgabe der realen Zielrepräsentation verglichen. Gegen die nuScenes-Annotation wird dagegen auf der Referenzskala ausgewertet. Dabei kommt Intersection over Union, *dt. Jaccard-Koeffizient* (IoU)_{max} zum Einsatz: Die dekodierten Karten werden mit sieben Schwellen zwischen 0,35 und 0,65 binarisiert, wobei je Klasse die beste IoU zählt. Mit der realen Zielrepräsentation erreicht der unveränderte Wahrnehmungspfad 0,6295 mIoU in der Segmentierung und 0,6858 mAP in der Detektion.

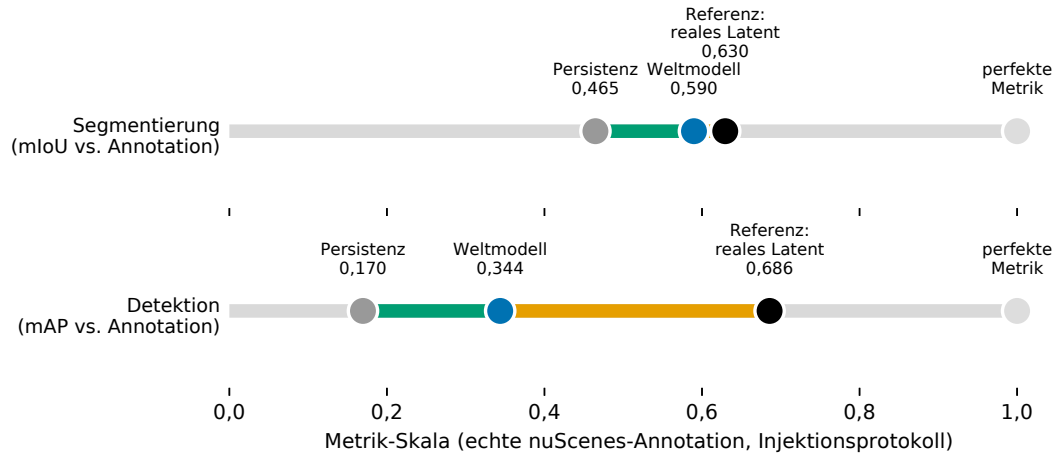


Abbildung 5.1: Persistenz, Weltmodell und Referenz mit realer Zielrepräsentation auf der Referenzskala, gemessen gegen die nuScenes-Annotation (Vollvalidierung).

Tabelle 5.1: Bezugsgrößen der Evaluierung. Segmentierung: mIoU; Proxy-Skala auf 5 743 Validierungsfenstern, Referenzskala (IoU_{max} gegen die nuScenes-Annotation) auf der vollen Validierungsmenge. Detektion: mAP auf der vollen Validierungsmenge.

Strang	Größe	Persistenz	reale Zielrepräsentation
Segmentierung (Proxy)	mIoU	0,5329	—
Segmentierung (Referenz)	mIoU	0,4647	0,6295
Detektion (Referenz)	mAP	0,1696	0,6858

Zur Einordnung kleiner Unterschiede dient die in Abschnitt 4.4 bestimmte Seed-Streuung. Als Grenzwerte werden 0,014 mIoU für die Segmentierung und konservativ

0,0088 mAP für die Detektion verwendet. Unterschiede darunter gelten als *Seed-Variabilität*. Bei den späteren Kopfadaptationen wird die Streuung des Kopftrainings separat bestimmt, da das Weltmodell unverändert bleibt.

5.2 Loss-Ablation und Modellvarianten

In der Loss-Ablation wird jeweils ein Term der Zielfunktion verändert, während die übrigen Gewichte auf der Ausgangskonfiguration bleiben. Zunächst werden die Varianten auf der Proxy-Skala verglichen. Ausgewählte Konfigurationen werden anschließend auf allen 5 743 Validierungsfenstern ausgewertet (Abbildung 5.2).

Mit dem Streuungsterm steigt das Streuungsverhältnis von 0,81 auf 0,99, ohne dass sich die mIoU messbar verändert. Einen ähnlichen Effekt zeigt der Wechsel auf Smooth-L1 (Abbildung 5.3). Beim Mittelwertterm ergibt sich kein messbarer Vorteil. Kosinus- und Gradiententerm verschlechtern dagegen das Streuungsverhältnis, während eine hohe SSIM-Gewichtung die mIoU von 0,687 auf 0,670 senkt. Als Segmentierungskonfiguration wird deshalb Smooth-L1 mit Streuungsterm verwendet. Auf allen 5 743 Validierungsfenstern erreicht diese Variante 0,6946 mIoU gegenüber 0,6920 der Sechs-Term-Konfiguration. Der Unterschied liegt innerhalb der Seed-Variabilität.

In der Detektion liegt die Sechs-Term-Konfiguration in zwei Trainingsläufen um 0,0174 beziehungsweise 0,0187 mAP über der minimalen Konfiguration. Beide Unterschiede überschreiten die Schwelle von 0,0088 mAP. Zusätzliche Loss-Terme bringen hier damit einen messbaren Vorteil, während sich dieser Effekt in der Segmentierung nicht zeigt. Ob die Rangfolge der Segmentierungsvarianten auch gegen die nuScenes-Annotation erhalten bleibt, wird in Abschnitt 5.4 geprüft.

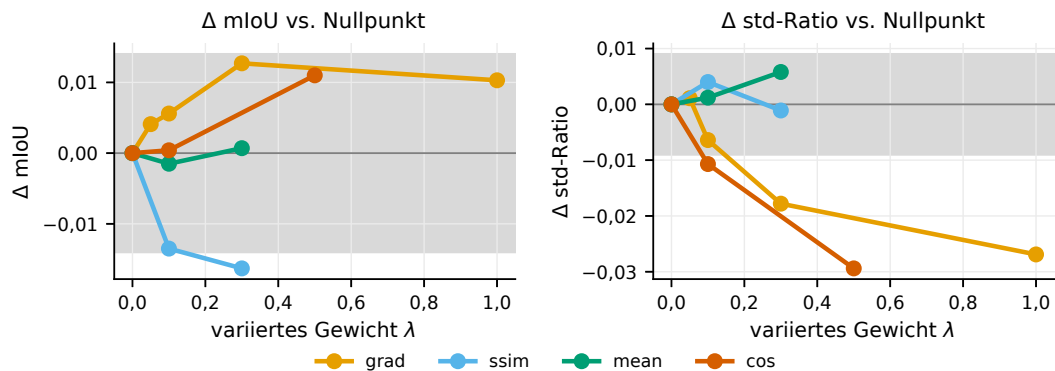


Abbildung 5.2: Einfluss der einzelnen Loss-Gewichte auf der Proxy-Skala (Teilstichprobe mit 300 Fenstern, ein Lauf je Gewichtseinstellung). Grau hinterlegt: Streuungsband der Seed-Variabilität.

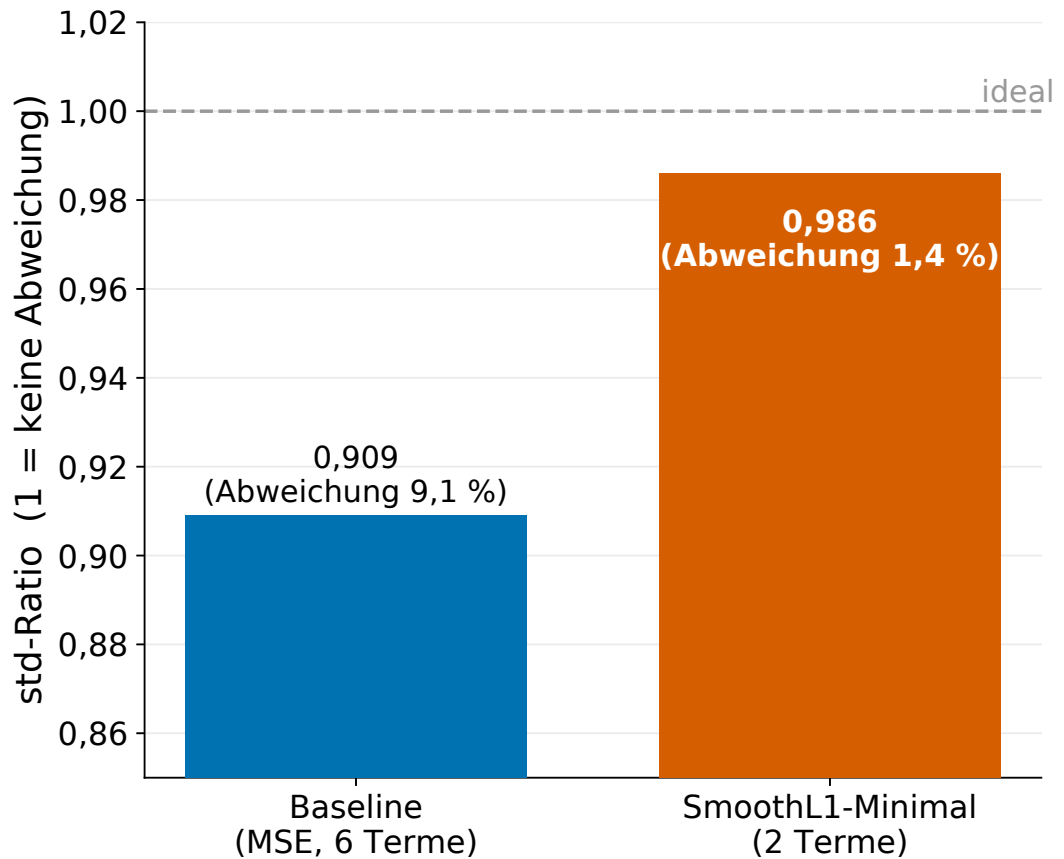


Abbildung 5.3: Streuungsverhältnis vor und nach dem Wechsel der Rekonstruktionsdistanz auf Smooth-L1 (Ein-Schritt-Vorhersage, Vollvalidierung).

Weitere Modellvarianten. Neben der Zielfunktion wurden weitere Modellvarianten untersucht (Abbildung 5.4). Weder Ego-Konditionierung noch ein vierter Kontextframe, ein Exponential Moving Average (EMA), acht statt vier Transformer-Schichten oder ein festes Residual verbessern die mIoU messbar. Schlechtere Werte ergeben sich bei direkten Modulationen der BEV-Repräsentation und beim Decoder-Task-Loss. Mehr Modellkapazität bringt im untersuchten Größenbereich ebenfalls keinen messbaren Vorteil. Die vollständigen Werte stehen in Abschnitt A.3 und Tabelle A.1.

Das Weltmodell übertrifft in der Segmentierung beide Persistenz-Baselines. Gegenüber der ego-kompensierten Variante beträgt der Vorsprung 0,067 mIoU nach einem Schritt und 0,035 nach vier Schritten. Die Ego-Kompensation verringert damit den Abstand zum Weltmodell gegenüber der naiven Persistenz. Über vier autoregressive Schritte sinkt die mIoU des Weltmodells von 0,695 auf 0,477, während das Streuungsverhältnis nahe eins bleibt. Abbildung 5.5 und Abbildung 5.6 zeigen dazu eine statische und eine dynamische Szene.

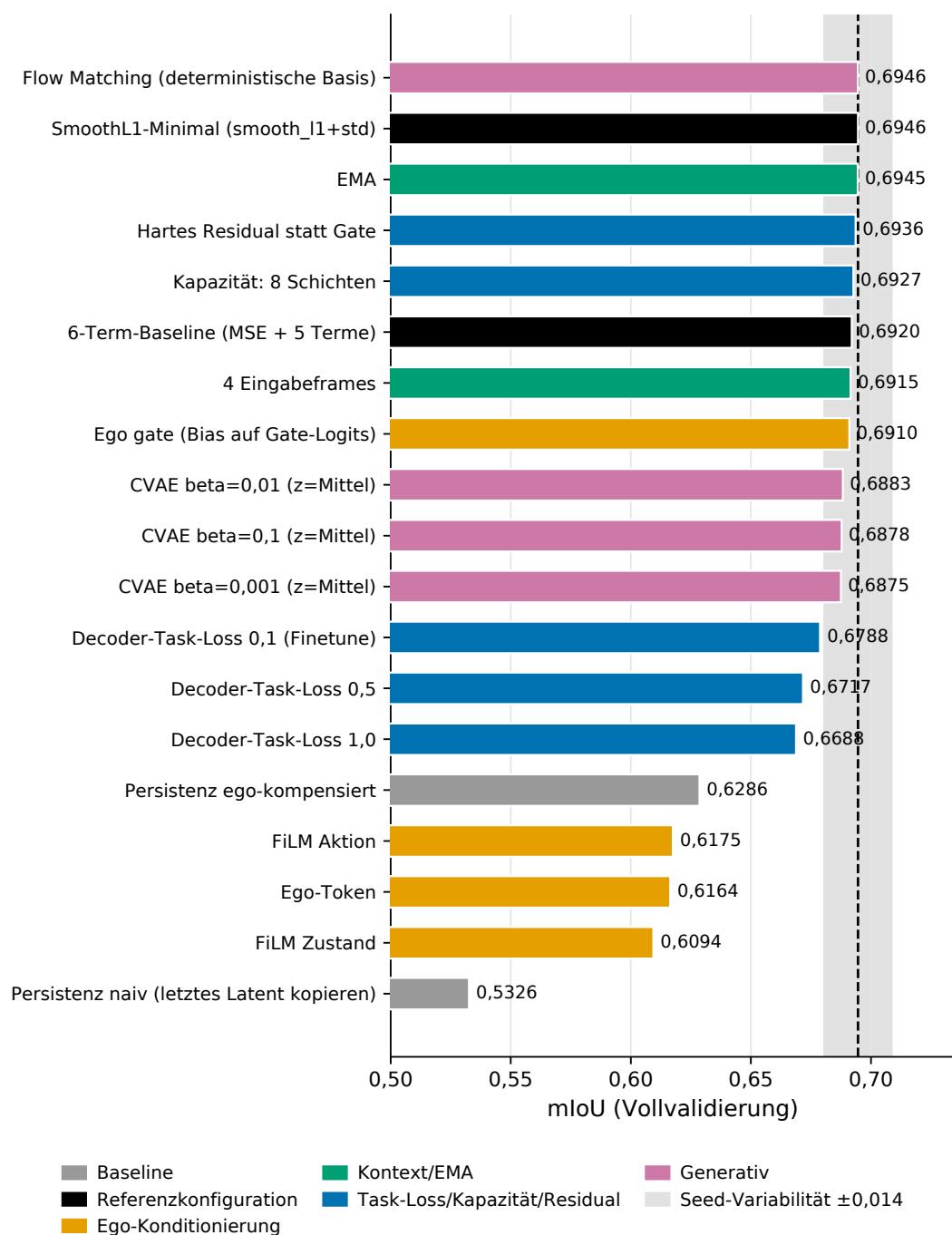


Abbildung 5.4: mIoU der untersuchten Modellvarianten auf 5 743 Validierungsfenstern. Grau markiert ist die Seed-Variabilität der Referenz.

In der statischen Szene bleibt die Vorhersage nahezu deckungsgleich mit der Realität. In der dynamischen driftet sie von 0,31 bei einem Schritt auf 0,18 bei vier Schritten ab.

Abbildung 5.7 veranschaulicht die Gate-Werte an mehreren Beispielszenen. In der ausgewählten statischen Szene liegen sie überwiegend nahe null, in der dynamischen

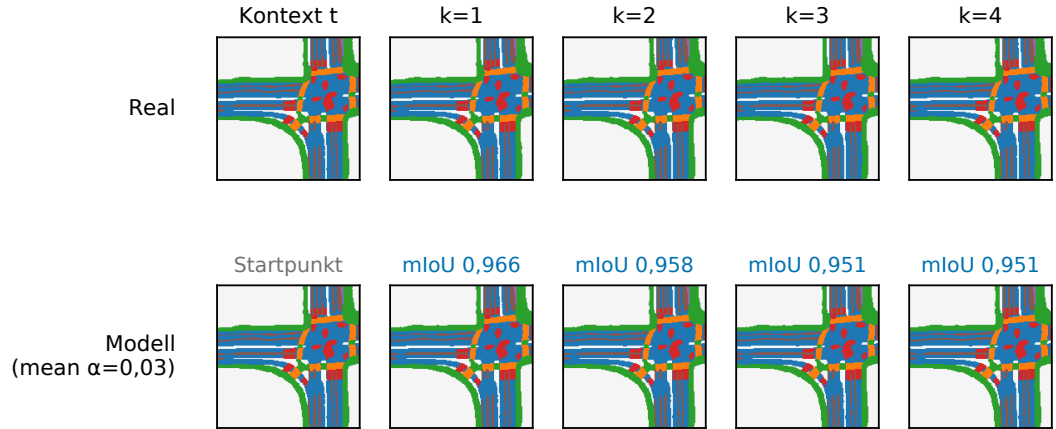


Abbildung 5.5: Autoregressiver Rollout in einer statischen Szene: reale Sequenz (oben) gegen Vorhersage (unten), letzter Kontextframe und vier Rollout-Schritte. Szenenauswahl automatisch über das Minimum der mittleren Gate-Aktivierung ($\alpha \approx 0,03$), nicht nach Ergebnisqualität.

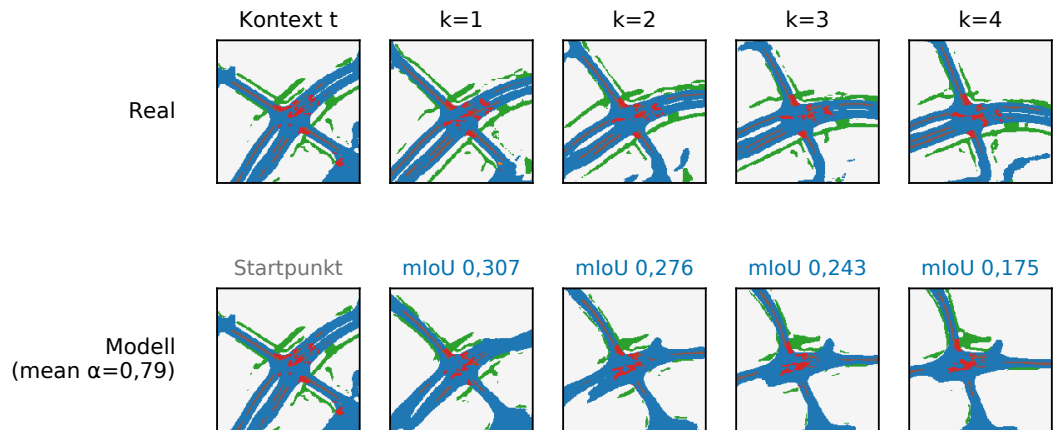


Abbildung 5.6: Autoregressiver Rollout in einer dynamischen Szene (Auswahl über das Maximum der mittleren Gate-Aktivierung, $\alpha \approx 0,79$): Die Vorhersage verliert die bewegten Strukturen über den Horizont zunehmend.

Szene dagegen deutlich höher. Niedrige Werte gewichten die jüngste Kontextrepräsentation stärker, höhere Werte die Transformer-Vorhersage.

Zwischen den Klassen bestehen deutliche Unterschiede (Abbildung 5.8). Mit 0,897 mIoU erreicht die befahrbare Fläche den höchsten Wert, gefolgt von Gehwegen mit 0,734 und Haltelinien mit 0,555. Besonders dünne Strukturen reagieren empfindlich auf räumliche Verschiebungen. Zwischen zwei aufeinanderfolgenden realen Repräsentationen liegt die mIoU bei 0,53. Auch acht Transformer-Schichten verbessern das Ergebnis nicht messbar. Abschnitt 5.4 bewertet die Varianten anschließend gegen die nuScenes-Annotation.

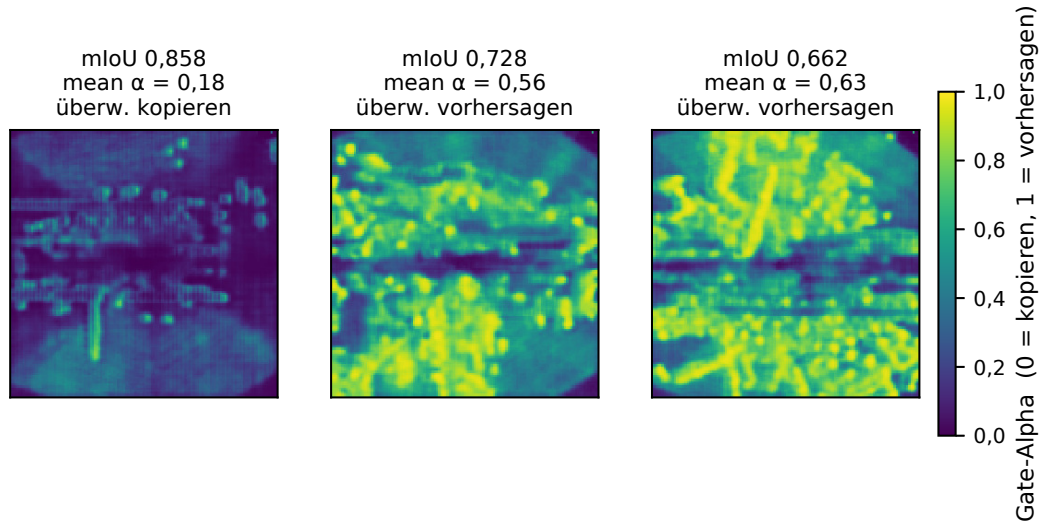


Abbildung 5.7: Gelernte Gate-Karten α für drei Beispielszenen. Kleine Werte gewichten die jüngste Kontextrepräsentation stärker, große Werte die Transformer-Vorhersage.

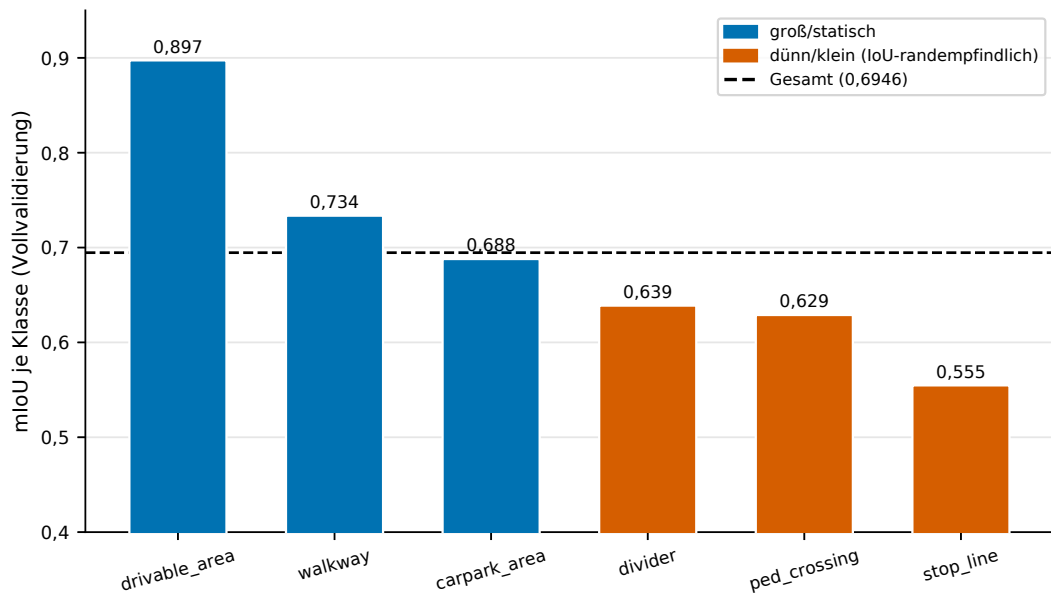


Abbildung 5.8: mIoU der Minimal-Konfiguration je Klasse auf der vollständigen Validierungsmenge.

5.3 Generative Vorhersageköpfe

Untersucht wird, wie stark sich die Vorhersagen von CVAE und Flow Matching unterscheiden und wie sich dies auf die Segmentierungsmetrik auswirkt.

Bei den CVAE-Varianten liegen die mIoU-Werte nahe an der deterministischen Referenz, während sich die erzeugten Vorhersagen kaum unterscheiden (Abbildung 5.9).

Zwischen acht Stichproben unterscheiden sich nur 0,06 Prozent der Maskenzellen, und auch die Streuung der Repräsentationen bleibt gering. Weitere β -Werte sind in Abschnitt A.4 dargestellt.

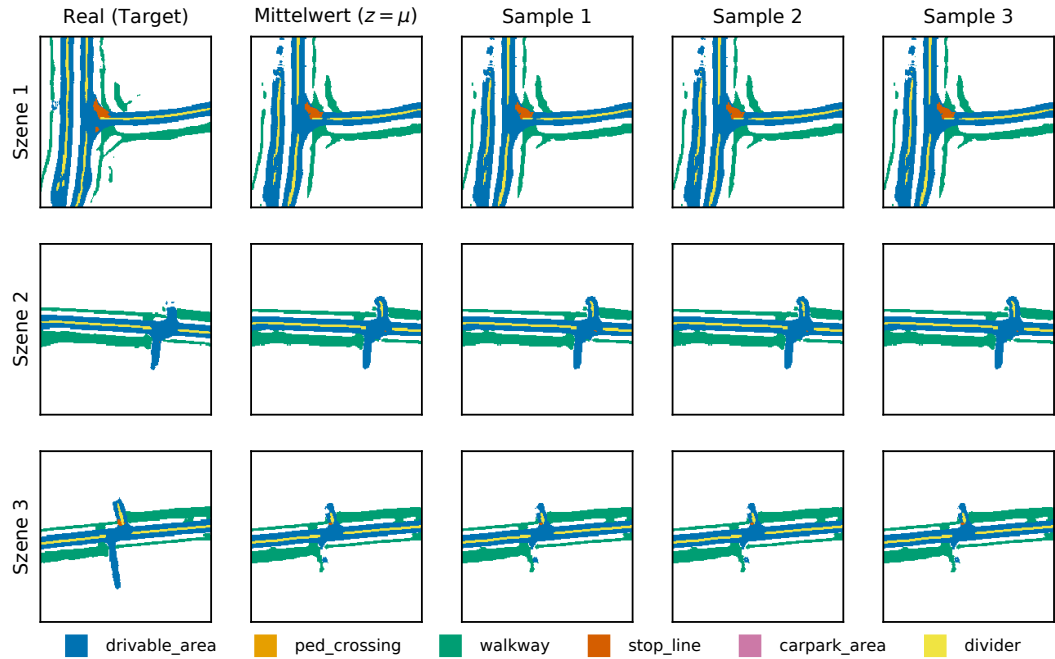


Abbildung 5.9: CVAE-Kopf ($\beta = 0,01$): Mittelwert und drei dargestellte Stichproben unterscheiden sich kaum.

Mehr Variation zeigt Flow Matching (Abbildung 5.10). Eine einzelne Stichprobe liegt im Mittel 0,044 mIoU unter der deterministischen Vorhersage. Bereits ein Euler-Schritt liefert praktisch dasselbe Ergebnis wie zehn Schritte.

Mit steigender Zahl der Stichproben nimmt die Best-of- K -mIoU von 0,6450 bei $K = 1$ auf 0,6875 bei $K = 16$ zu. Auf derselben Teilstichprobe mit 300 Fenstern erreicht die deterministische Vorhersage 0,6896. Nach Residual-Skalierung und längerem Training liegt Best-of-8 in einem separaten Versuch numerisch 0,010 mIoU darüber (Abbildung 5.11). Für die Auswahl wird allerdings das reale Ziel verwendet. Best-of- K zeigt daher nur, ob unter mehreren Stichproben eine Vorhersage mit höherer Übereinstimmung zum Ziel enthalten ist.

Im autoregressiven Rollout bleibt dieser Unterschied bestehen: Eine Flow-Stichprobe liegt dort etwa 0,045 mIoU unter der deterministischen Vorhersage.

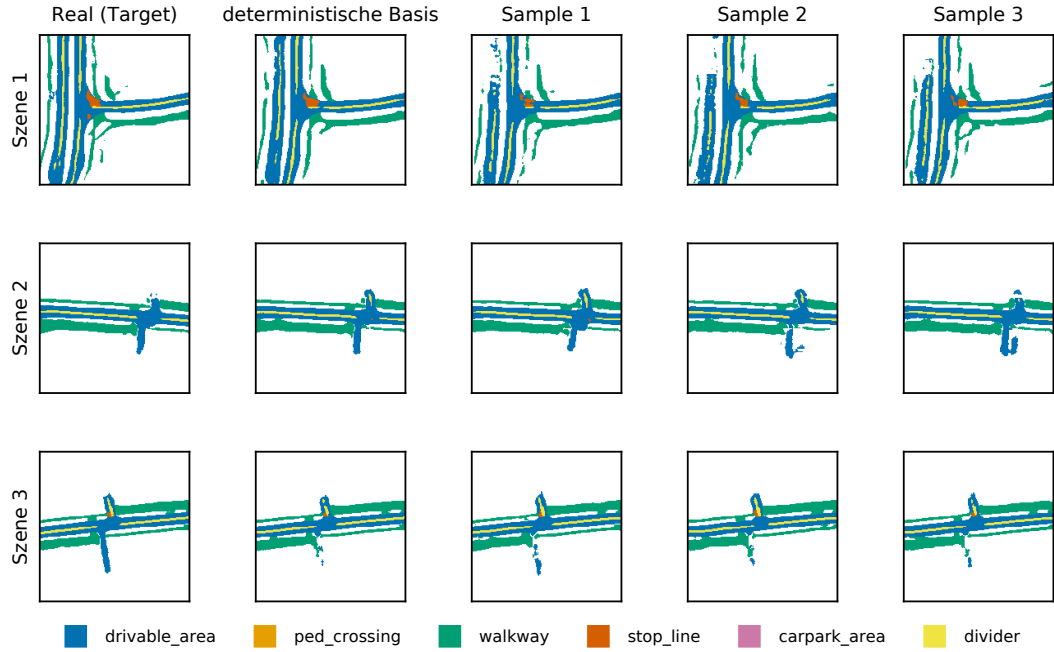


Abbildung 5.10: Flow-Matching-Kopf: dieselbe Szene mit sichtbarer Variation zwischen den Stichproben.

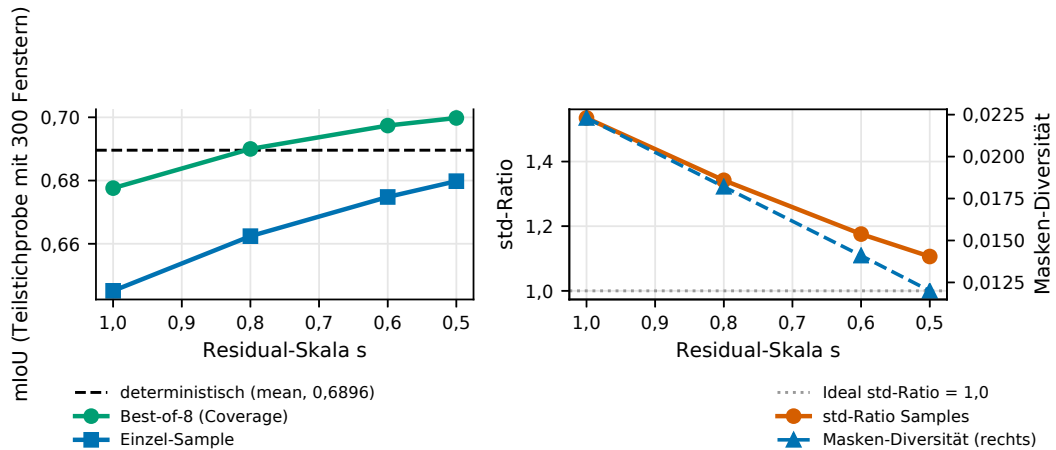


Abbildung 5.11: Einfluss der Residual-Skalierung auf die Flow-Matching-Stichproben. Best-of-8 verwendet das bekannte Ziel zur Auswahl und ist daher eine Auswertung mit Kenntnis des Zielzustands.

5.4 Bewertung gegen die nuScenes-Annotation

Bisher wurde die Segmentierung überwiegend über die Übereinstimmung zwischen Vorhersage und realer Zielrepräsentation nach dem Decoder bewertet. Nun folgt die Auswertung direkt gegen die nuScenes-Annotation. Bei der Detektion wurde diese Referenzskala bereits für die vorherigen Ergebnisse verwendet.

Von der Persistenz zum Weltmodell steigt die mAP von 0,1696 auf 0,3438, während die Referenz mit realer Zielrepräsentation 0,6858 erreicht (Abbildung 5.12). Damit liegt das Weltmodell bei mehr als dem Doppelten der Persistenz, aber weiterhin deutlich unter der Referenz mit realer Zielrepräsentation. Ergänzend zeigt Abbildung 5.12 den nuScenes Detection Score (NDS), der neben der Detektionsgüte auch Lage- und Attributfehler berücksichtigt.

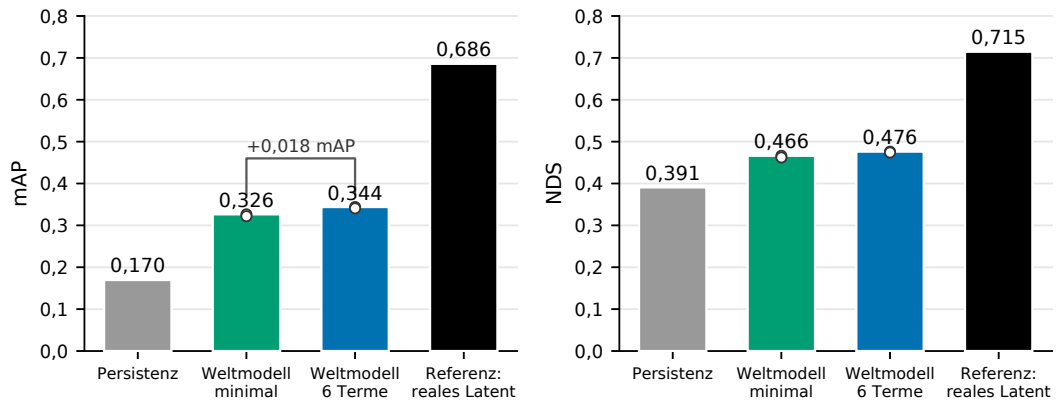


Abbildung 5.12: Detektionsstrang auf der Referenzskala (Injektion, Vollvalidierung): Persistenz, Weltmodell und Referenz mit realer Zielrepräsentation in mAP und NDS.

Auf der Referenzskala erreicht die Segmentierung 0,4647 mIoU mit der Persistenz, 0,5898 mit dem Weltmodell und 0,6295 mit der realen Zielrepräsentation (Abbildung 5.13). Auch die Rangfolge der Loss-Ablation bleibt erhalten: Die minimale Konfiguration erreicht 0,5898 und die Sechs-Term-Konfiguration 0,5882 mIoU. Mit 0,0016 mIoU liegt dieser Unterschied innerhalb der Seed-Variabilität.

Auch auf Klassenebene bleibt die Referenz mit realer Zielrepräsentation unter einer perfekten Segmentierung. Bei den Haltelinien werden beispielsweise 0,4746 mIoU mit dem Weltmodell und 0,5114 mit realer Zielrepräsentation erreicht. Dabei beschreibt die Proxy-mIoU von 0,6946 die Übereinstimmung mit der dekodierten Ausgabe der realen Zielrepräsentation, während die Referenz-mIoU von 0,5898 direkt gegen die nuScenes-Annotation gemessen wird. Beide Werte beruhen daher auf unterschiedlichen Vergleichszielen.

Metrikabstände. Abbildung 5.14 stellt das Weltmodell der Referenz mit realer Zielrepräsentation und der perfekten Metrik gegenüber. In der Segmentierung beträgt der Abstand zwischen der Referenz mit realer Zielrepräsentation und der perfekten Metrik 0,371 mIoU. Weitere 0,040 mIoU liegen zwischen Weltmodell und Referenz mit realer Zielrepräsentation.

Bei der Detektion liegen 0,314 mAP zwischen der Referenz mit realer Zielrepräsentation und der perfekten Metrik sowie weitere 0,342 mAP zwischen Weltmodell und Referenz. Beide Abstände sind damit ähnlich groß. Abbildung 5.14 zeigt ausschließlich Metrikabstände und keine kausal getrennten Fehlerquellen.

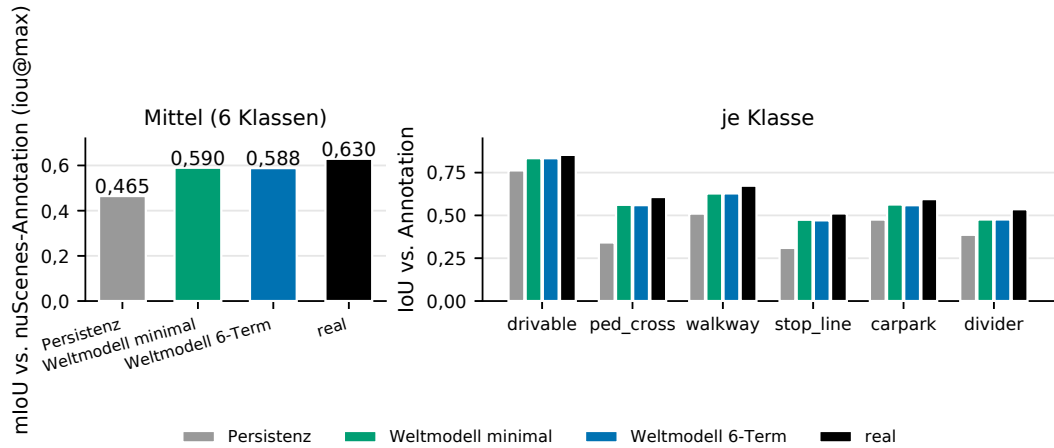


Abbildung 5.13: Persistenz, Weltmodell und Referenz mit realer Zielrepräsentation sowie die Ergebnisse der einzelnen Klassen gegen die nuScenes-Annotation.

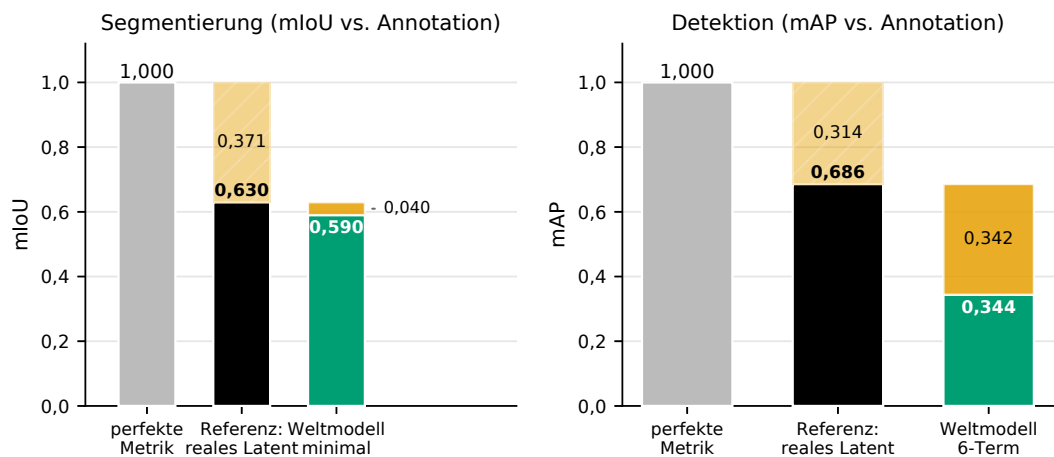


Abbildung 5.14: Abstand der Referenz mit realer Zielrepräsentation zur perfekten Metrik und zusätzlicher Abstand des Weltmodells zu dieser Referenz. Die Darstellung beschreibt Metrikabstände, keine kausal getrennten Fehleranteile.

Zwischen Weltmodell und Referenz mit realer Zielrepräsentation liegen in der Segmentierung 0,040 mIoU, in der Detektion dagegen 0,342 mAP. Auch die Loss-Ablation unterscheidet sich zwischen den Aufgaben: Zusätzliche Loss-Terme verbessern die Detektion, nicht aber die Segmentierung.

Gewichtung im Weltmodell. Zunächst wurde geprüft, ob Änderungen am Training des Weltmodells die annotationsbasierte Leistung verbessern. Untersucht wurden eine zellgewichtete Rekonstruktions-Loss-Funktion sowie Datenlader mit stärkerer Gewichtung seltener Klassen oder dynamischer Szenen. Auf der Proxy-Skala liegen einzelne Varianten leicht über ihrer Referenz, die Unterschiede bleiben jedoch innerhalb der Seed-Variabilität. Gegen die nuScenes-Annotation ergibt sich ebenfalls kein messbarer Vorteil. Ein Teil dieser Läufe wurde früher gestoppt, sodass daraus

nur abgeleitet werden kann, dass in diesen Versuchen keine klare Verbesserung nachweisbar war.

Die zellgewichtete Loss-Funktion erhöht das Streuungsverhältnis bis auf 0,9975, ohne die Referenz-mIoU messbar zu verbessern. Außerdem zeigt der Vergleich beider Skalen, dass eine bessere Übereinstimmung mit der dekodierten Ausgabe der realen Zielrepräsentation nicht zwingend zu einer besseren mIoU gegen die Annotation führt.

Auch die übrigen Änderungen am Weltmodell verbessern die annotationsbasierte mIoU nicht messbar. Auf der Proxy-Skala bleibt 0,6946 der beste Wert, während gegen die nuScenes-Annotation 0,5898 mIoU erreicht werden. Selbst die reale Zielrepräsentation kommt mit demselben Kopf nur auf 0,6295 mIoU. Deshalb wird im nächsten Schritt geprüft, ob sich die Auswertung vorhergesagter Repräsentationen durch eine Anpassung des nachgelagerten Kopfes verbessern lässt. Das Weltmodell bleibt dabei unverändert.

Adaptation des Segmentierungskopfs. Mit unverändertem Weltmodell erreicht der adaptierte Segmentierungskopf über vier Trainingsläufe $0,6008 \pm 0,0005$ mIoU gegenüber 0,5898 des ursprünglichen Kopfes. Der Gewinn von $0,0110 \pm 0,0005$ mIoU tritt in allen vier Läufen auf. Auf realen Repräsentationen bleibt die Leistung nahezu unverändert und erreicht in einem beispielhaften Lauf 0,6293 statt 0,6295 mIoU. Bei den Haltelinien steigt die mIoU in diesem Lauf um 0,0325.

Adaptation des Detektionskopfs. Mit der ersten adaptierten Fassung steigt die mAP auf vorhergesagten Repräsentationen von 0,3438 auf 0,4329, während sie auf realen Repräsentationen von 0,6858 auf 0,6603 fällt. Eine zweite Fassung ergänzt eine Beimischung realer Repräsentationen, längeres Training und eine Checkpoint-Auswahl. Über drei Trainingsläufe erreicht sie im Mittel $0,4314 \pm 0,0021$ mAP auf Vorhersagen und $0,6720 \pm 0,0008$ auf realen Repräsentationen (Abbildung 5.16). Gegenüber dem ursprünglichen Kopf verbessert sich die mAP auf Vorhersagen in jedem Lauf, während auf realen Repräsentationen ein Rückstand von rund 0,014 mAP bleibt.

Abbildung 5.17 zeigt ausgewählte Beispiele. Weitere Szenen stehen in Abschnitt A.5, die vollständigen Werte in Tabelle A.2.

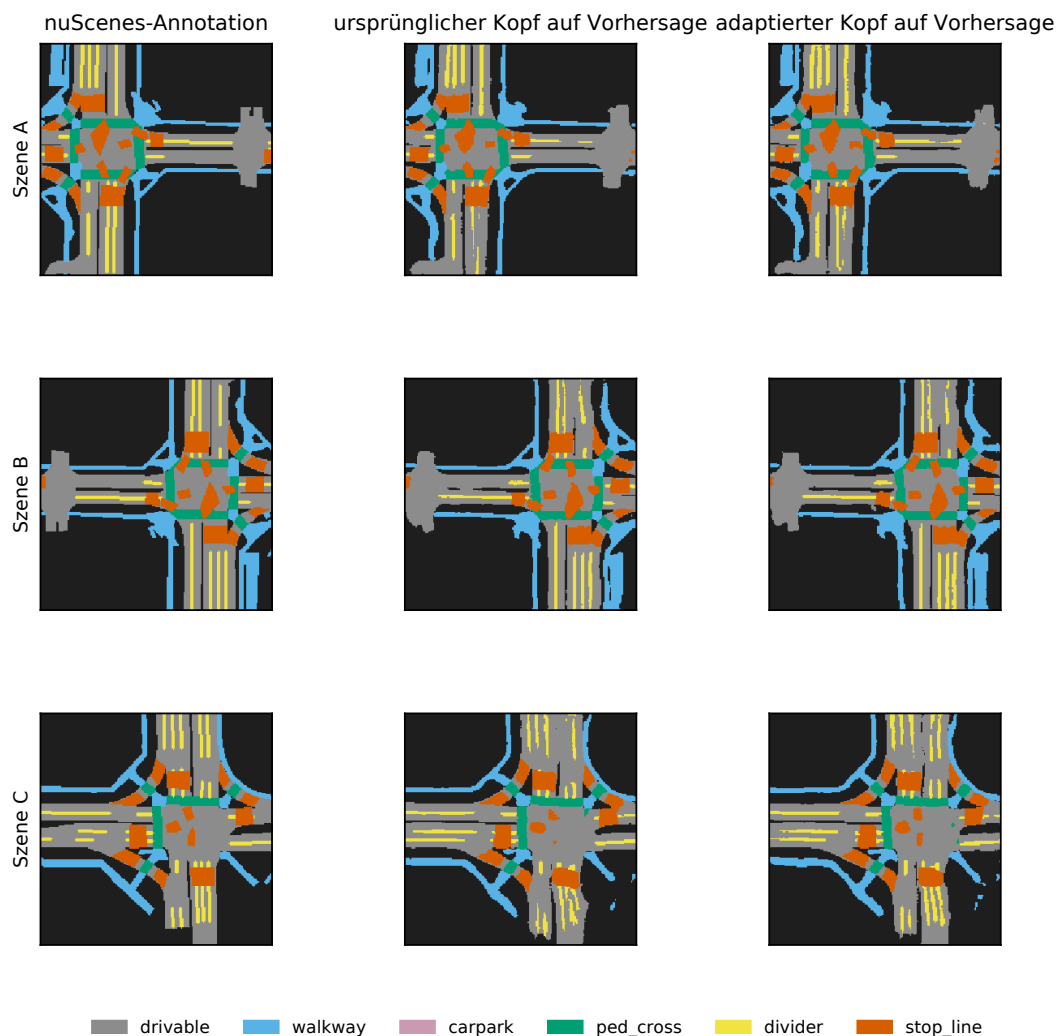


Abbildung 5.15: Identische Weltmodell-Vorhersagen mit ursprünglichem und adaptiertem Segmentierungskopf, bewertet gegen die nuScenes-Annotation.

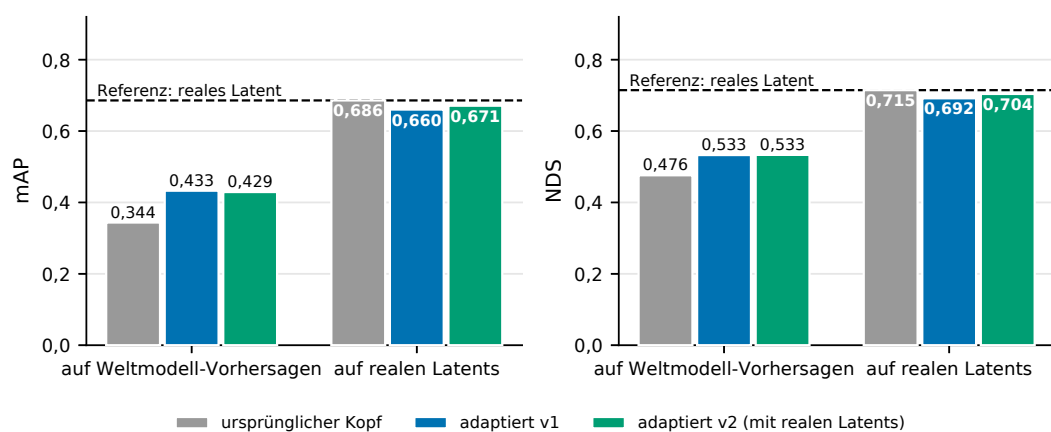


Abbildung 5.16: Ursprünglicher Kopf und adaptierte Detektionsköpfe auf Weltmodell-Vorhersagen und realen Repräsentationen.

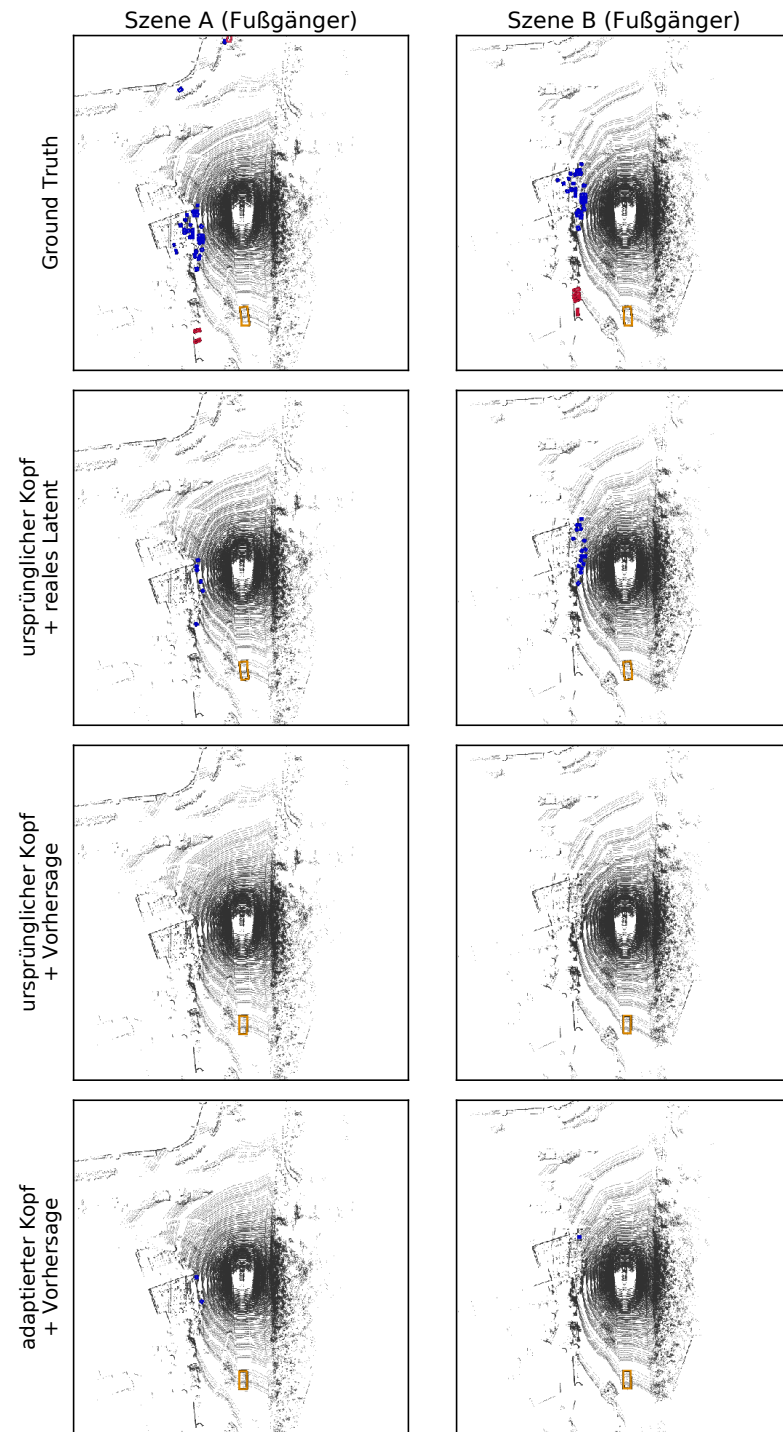


Abbildung 5.17: *Detektionen aus injizierten Vorhersagerepräsentationen mit eingefrorenem und adaptiertem Kopf. Die gezeigten Szenen wurden nach maximaler Differenz zwischen beiden Köpfen ausgewählt, um den Effekt sichtbar zu machen; sie sind nicht repräsentativ für den Durchschnittsfall.*

Einfluss der Kopfadaptation. Bezogen auf die Mittelwerte der wiederholten Trainingsläufe reduziert die Kopfadaptation den Abstand zur Referenz mit realer Zielrepräsentation um rund 28 Prozent in der Segmentierung und 26 Prozent in der Detektion. Während die untersuchten Änderungen am Weltmodell keinen vergleichbaren Gewinn zeigen, verbessert ein auf vorhergesagte Repräsentationen angepasster Kopf in beiden Strängen die Zielmetrik.

5.5 Ressourcenbedarf

Die Inferenzzeit wurde für einen einzelnen Vorhersageschritt bei Batchgröße eins und gemischter Präzision auf einer NVIDIA TITAN RTX gemessen. Jede Messserie umfasst 30 Aufwärm- und 300 Messiterationen und wurde dreimal wiederholt. Zwischen den drei Wiederholungen unterscheiden sich die Mittelwerte je Variante um weniger als 0,5 ms. Da der GPU-Takt nicht manuell festgelegt wurde, gelten die Laufzeiten für die verwendete Workstation-Konfiguration.

Für den Grafikspeicher wird im Folgenden die Bezeichnung Video Random Access Memory (VRAM) verwendet.

Tabelle 5.2: *Parameterzahl, Speicherbedarf und Inferenzzeit der Modellvarianten bei Batchgröße eins auf einer NVIDIA TITAN RTX.*

Variante	Parameter	Gewichte (16 Bit) [MB]	VRAM [MB]	Zeit [ms]
Segmentierung, minimal	6,05 M	11,5	147	$4,41 \pm 0,32$
Segmentierung, 8 Schichten	9,21 M	17,6	165	$7,10 \pm 0,04$
Detektion, 6 Terme	6,06 M	11,6	249	$11,45 \pm 0,07$
Flow-Kopf (gesamt)	12,57 M	24,0	266	14,02
CVAE-Kopf	6,42 M	12,3	149	$4,58 \pm 0,01$
BEVFusion-Encoder	—	—	—	$172,78 \pm 1,16$

Gemessen wird nur die zusätzliche Latenz des Weltmodells. Für einen Vorhersageschritt benötigt die minimale Segmentierungsvariante 4,4 ms und die Detektionsvariante 11,5 ms. Auf derselben Workstation liegt die Laufzeit des BEVFusion-Encoders bei rund 173 ms.

Mit acht Transformer-Schichten steigt die Latenz auf 7,1 ms, ohne dass sich die mIoU messbar verbessert. Ein Euler-Schritt des Flow-Kopfs benötigt rund 14,0 ms. Beim GPU-Speicher liegt der höchste gemessene Wert bei 266 MB. Angegeben ist jeweils der Spitzenwert des durch PyTorch allokierten Speichers während eines Vorwärtsdurchlaufs bei Batchgröße eins. Er umfasst die 32-Bit-Modellgewichte, die Eingaberepräsentationen und die Zwischenaktivierungen des Weltmodells, nicht jedoch den Speicher der Laufzeitumgebung oder den separat ausgeführten BEVFusion-Encoder. Je nach Variante dauert das Training der Weltmodelle elf bis 25 GPU-Stunden, während die Kopfadaptationen etwa eine bis vier GPU-Stunden benötigen.

Abbildung 5.18 stellt Vorhersagequalität, Latenz und GPU-Speicher der Segmentierungsmodelle gegenüber. Unter den direkt vergleichbaren Varianten bringt der höhere Ressourcenbedarf keinen messbaren Qualitätsgewinn. Nicht dargestellt ist der Flow-Kopf, da seine Stichproben nur auf der Teilstichprobe mit 300 Fenstern ausgewertet wurden. Seine Ressourcenwerte sind in Tabelle 5.2 angegeben.

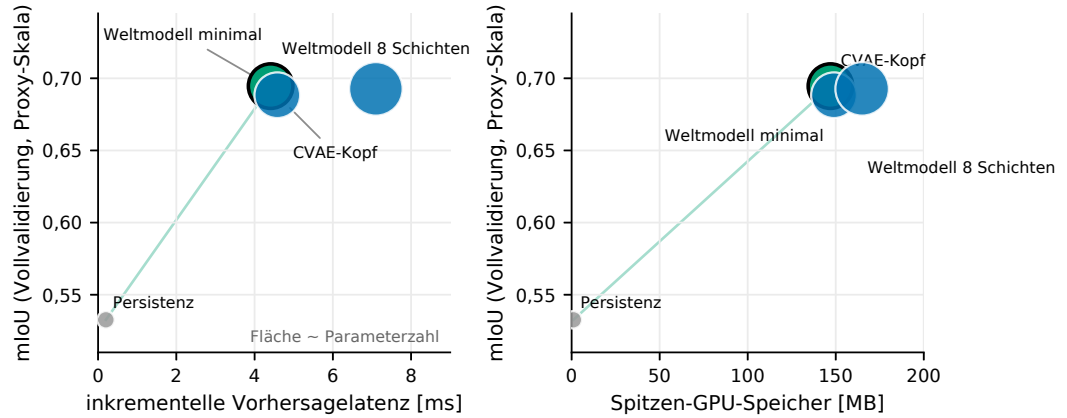


Abbildung 5.18: *Qualität, Latenz und Spitzen-GPU-Speicher der Modellvarianten des Segmentierungsstrangs (Vollvalidierung). Der Flow-Kopf ist nicht enthalten (Tabelle 5.2).*

5.6 Empfohlene Konfigurationen und Limitationen

Empfohlene Konfigurationen. Als Segmentierungskonfiguration wird Smooth-L1 mit Streuterm verwendet. Mit dem ursprünglichen Kopf erreicht diese Variante 0,5898 mIoU gegen die nuScenes-Annotation, nach der Kopfadaptation über vier Trainingsläufe $0,6008 \pm 0,0005$ mIoU.

Die resultierende Zielfunktion besteht aus einem Vorhersagefehler und einem Term für die Statistik der Repräsentationen. Eine ähnliche Grundstruktur findet sich bei LeWorldModel [5], allerdings werden dort Encoder und Prädiktor gemeinsam trainiert und die Verteilung der latenten Repräsentationen anders reguliert.

Im Detektionsstrang wird dagegen die Sechs-Term-Konfiguration verwendet. Auf vorhergesagten Repräsentationen erreicht der ursprüngliche Kopf 0,3438 mAP. Nach der zweiten Kopfadaptation steigt der Mittelwert über drei Trainingsläufe auf $0,4314 \pm 0,0021$ mAP. Auf realen Repräsentationen erreicht der adaptierte Kopf $0,6720 \pm 0,0008$ mAP gegenüber 0,6858 mit dem ursprünglichen Kopf.

In der Segmentierung genügen damit zwei Loss-Terme, während zusätzliche Terme in der Detektion einen messbaren Vorteil bringen.

Limitationen. Eine Einschränkung liegt in der Anzahl wiederholter Trainingsläufe. Zur Bestimmung der Seed-Streuung wurden drei Läufe in der Segmentierung und

zwei in der Detektion verwendet, während die endgültigen Kopfadaptationen mit vier beziehungsweise drei Seeds wiederholt wurden. Bei der Beimischung realer Repräsentationen wurde nur ein Anteil von 20 Prozent untersucht, und die erste Fassung des Detektionskopfes wurde nur einmal trainiert. Einige Gewichtungsvarianten endeten zudem früher als ihre Referenz, sodass bei diesen Varianten vor allem die Richtung der beobachteten Änderung bewertet werden kann.

Auch der Geltungsbereich ist begrenzt. Sämtliche Ergebnisse basieren auf nuScenes und den verwendeten BEVFusion-Konfigurationen mit eingefrorenen BEV-Repräsentationen. Durch die Schlüsselbilder ist eine Schrittweite von einer halben Sekunde vorgegeben. Eine Übertragung auf andere Datensätze, Wahrnehmungsmodelle oder Zeitschritte müsste daher separat untersucht werden. Zudem können Proxy- und annotationsbasierte Bewertung zu unterschiedlichen Ergebnissen führen.

Ein autoregressiver Detektions-Rollout wurde nicht untersucht. Bei den Ressourcennmessungen wurde nur die zusätzliche Vorhersagelatenz auf einer Workstation-GPU erfasst. Aussagen zu Gesamtlatenz und Energieverbrauch auf eingebetteter Zielhardware sind daher mit den vorliegenden Messungen nicht möglich.

6 Zusammenfassung und Ausblick

Im Mittelpunkt dieser Arbeit steht die Vorhersage zukünftiger BEV-Repräsentationen mit einem kompakten Transformer von rund sechs Millionen Parametern. Aus drei vergangenen Repräsentationen der jeweiligen eingefrorenen BEVFusion-Konfiguration wird die folgende Repräsentation vorhergesagt und auf dem nuScenes-Datensatz für Segmentierung und 3D-Objektdetektion bewertet. Verglichen wurden unterschiedliche Zielfunktionen und Modellvarianten sowie generative Erweiterungen und angepasste Wahrnehmungsköpfe.

Die vier Forschungsfragen lassen sich auf Grundlage der Ergebnisse wie folgt beantworten.

Das Weltmodell übertrifft die Persistenz-Baselines in beiden Wahrnehmungsaufgaben (Frage 1). In der Segmentierung liegt es sowohl über der naiven als auch über der ego-kompensierten Persistenz und behält den Vorsprung im autoregressiven Rollout. Bei der Detektion erreicht das Weltmodell mehr als das Doppelte der mAP der verwendeten Persistenz.

Die Zielfunktion wirkt in Segmentierung und Detektion unterschiedlich (Frage 2). In der Segmentierung genügen Smooth-L1 und ein Streuungsterm, während die vollständige Sechs-Term-Konfiguration keinen messbaren Vorteil bringt. In der Detektion erhöht sie die mAP dagegen in zwei Trainingsläufen um 0,0174 beziehungsweise 0,0187.

Keine der untersuchten Erweiterungen verbessert die Punktvorhersage messbar. Mit acht statt vier Transformer-Schichten steigt die Parameterzahl von rund 6,05 auf 9,21 Millionen und die gemessene Latenz von 4,4 ms auf 7,1 ms, ohne die Segmentierungsqualität zu verbessern. Beim CVAE unterscheiden sich die erzeugten Stichproben nur gering. Flow Matching erzeugt stärkere Unterschiede zwischen den Stichproben, erreicht mit einzelnen Vorhersagen jedoch eine niedrigere mIoU als das deterministische Modell. Mit wachsendem K steigt die Best-of- K -mIoU. Da die Auswahl dabei das bekannte Ziel verwendet, dient diese Auswertung nur der Analyse der erzeugten Varianten.

Zwischen Weltmodell und der Referenz mit realer Zielrepräsentation verbleibt in beiden Strängen ein messbarer Abstand (Frage 3). In der Segmentierung beträgt er 0,040 mIoU, in der Detektion 0,342 mAP. Von der Referenz mit realer Zielrepräsentation zur perfekten Metrik fehlen weitere 0,371 mIoU beziehungsweise 0,314 mAP. Diese Werte beschreiben Metrikabstände und keine kausal getrennten Fehlerquellen.

Im untersuchten Versuchsaufbau bringt die Anpassung der nachgelagerten Köpfe den größeren messbaren Gewinn (Frage 4). Bei unverändertem Weltmodell steigt die Segmentierung über vier Trainingsläufe von 0,5898 auf $0,6008 \pm 0,0005$ mIoU und die Detektion über drei Läufe von 0,3438 auf $0,4314 \pm 0,0021$ mAP. Änderungen an der Gewichtung des Weltmodelltrainings zeigen keinen vergleichbaren Vorteil. Auf realen Repräsentationen bleibt die Segmentierungsleistung nahezu unverändert,

während der angepasste Detektionskopf $0,6720 \pm 0,0008$ mAP gegenüber 0,6858 des ursprünglichen Kopfes erreicht.

DINO-Foresight und verwandte Ansätze sagen ebenfalls zukünftige Merkmale auf eingefrorenen Repräsentationen voraus und werten sie anschließend auf nachgelagerten Aufgaben aus [26], [32]. Bei LeWorldModel wird ein Vorhersagefehler mit einem Term für die Statistik der Repräsentationen kombiniert [5]. Eine ähnliche Grundstruktur genügt in dieser Arbeit für die Segmentierung, nicht jedoch für die Detektion. Auch bei den generativen Varianten verbessert die zusätzliche Variation durch Flow Matching die verwendete Punktmetrikt nicht [15], [40]. Kompakte, aufgabenrelevante Repräsentationen werden zudem in [28], [35] untersucht. Wegen unterschiedlicher Datensätze, Repräsentationen und Bewertungsmetriken sind direkte Leistungsvergleiche jedoch nicht möglich.

Verglichen mit dem BEVFusion-Encoder fällt die zusätzliche Vorhersagelatenz gering aus. Auf der verwendeten NVIDIA TITAN RTX benötigt ein Vorhersageschritt rund 4,4 ms für die Segmentierung und 11,5 ms für die Detektion, während der Encoder selbst bei rund 173 ms liegt. Erfasst wurde dabei nur die zusätzliche Vorhersagelatenz. Aussagen zur Gesamtlatenz oder zum Energieverbrauch auf Fahrzeughardware sind mit diesen Messungen nicht möglich.

Weitere Versuche sollten zunächst zusätzliche Klassen und unterschiedliche Anteile realer Repräsentationen bei der Kopfadaptation sowie einen autoregressiven Detektions-Rollout untersuchen. Bei generativen Vorhersagen wären außerdem Metriken sinnvoll, die mehrere mögliche Zukünfte berücksichtigen. Messungen auf eingebetteter Zielhardware könnten den Ressourcenbedarf hinsichtlich Latenz und Energieverbrauch ergänzen. Darüber hinaus könnte der Latentraum angepasst werden, etwa durch teilweises Mittrainieren des Encoders oder einen zusätzlichen kompakten aufgabenspezifischen Latentraum. Entscheidend ist dabei, ob der zusätzliche Aufwand einen messbaren Vorteil bringt und die Vergleichbarkeit mit den ursprünglichen Wahrnehmungsköpfen erhalten bleibt.

A Anhang

A.1 Gesamtübersicht der Weltmodell-Varianten

Tabelle A.1 führt alle im Segmentierungsstrang trainierten und ausgewerteten Weltmodell-Varianten dieser Arbeit zusammen, von den Persistenz-Baselines über die Loss-Konfigurationen bis zu den in Abschnitt 5.2 und Abschnitt 5.3 untersuchten Design- und Trainingsvarianten. Gemessen wird auf der Vollvalidierung (dekodierte Vorhersage gegen dekodierte reale Zielrepräsentation). Δ bezeichnet die Differenz zur minimalen Konfiguration (Smooth-L1 + Streuungsterm, 0,6946). Zur Einordnung wird die in Abschnitt 5.1 bestimmte Seed-Variabilität von 0,014 mIoU verwendet. Die Bewertung „innerhalb“ kennzeichnet Unterschiede innerhalb dieser Streuung. „Darunter“ steht für größere Verschlechterungen. Die Persistenz-Baselines wurden auf 5 467 Fenstern des Rollout-Protokolls bestimmt, die Variante mit vier Eingabeframes auf 5 651 Fenstern (größeres Kontextfenster, weniger gültige Sequenzen). Alle übrigen Werte beruhen auf den vollen 5 743 Fenstern.

Tabelle A.1: Alle Weltmodell-Varianten des Segmentierungsstrangs (Vollvalidierung, mIoU, Δ gegenüber der minimalen Konfiguration).

Gruppe	Variante	mIoU	Δ	Bewertung
Baseline	Persistenz naiv	0,5326	—	—
Baseline	Persistenz ego-kompensiert	0,6286	—	—
Loss	Sechs-Term-Baseline	0,6920	−0,0026	innerhalb
Loss	Minimal (Smooth-L1 + Streuung)	0,6946	—	Referenz
Ego	Gate-Bias	0,6910	−0,0036	innerhalb
Ego	Feature-wise Linear Modulation (FiLM) (action)	0,6175	−0,0771	darunter
Ego	Ego-Token	0,6164	−0,0782	darunter
Ego	FiLM (state)	0,6094	−0,0852	darunter
Kontext	4 Eingabeframes	0,6915	−0,0031	innerhalb
Kontext	Exponential Moving Average (EMA) (Shadow-Gewichte)	0,6945	−0,0001	innerhalb
Kontext	EMA-Lauf (Konsistenzprüfung)	0,6952	+0,0006	innerhalb
Training	Decoder-Task-Loss 0,1	0,6788	−0,0158	darunter
Training	Decoder-Task-Loss 0,5	0,6717	−0,0229	darunter
Training	Decoder-Task-Loss 1,0	0,6688	−0,0258	darunter
Architektur	Kapazität: 8 Schichten (9,21 M)	0,6927	−0,0019	innerhalb
Architektur	hartes Residual statt Gate	0,6936	−0,0010	innerhalb
Generativ	CVAE $\beta=0,001$ ($\mathbf{z}$ = Mittel)	0,6875	−0,0071	innerhalb
Generativ	CVAE $\beta=0,01$ ($\mathbf{z}$ = Mittel)	0,6883	−0,0063	innerhalb
Generativ	CVAE $\beta=0,1$ ($\mathbf{z}$ = Mittel)	0,6878	−0,0068	innerhalb
Generativ	Flow-Kopf (det. Mittel)	0,6946	± 0	identisch (Basispfad)

A.2 Kopfvarianten beider Stränge

Tabelle A.2 stellt die eingefrorenen und adaptierten Wahrnehmungsköpfe beider Stränge auf der Referenzskala gegenüber (Abschnitt 5.4). Jede Kombination aus Kopf und Eingangsverteilung ist eine eigene Zeile. Für den Detektionsstrang ist neben der mAP der NDS angegeben (Abschnitt 5.4). Die Tabellenwerte der adaptierten Köpfe stammen jeweils aus dem Lauf mit Startwert 42, damit mAP und NDS im Detektionsstrang aus demselben Lauf stammen. Die im Haupttext berichteten Werte sind dagegen Mittelwerte über mehrere Trainingsläufe und können deshalb von diesen Einzelwerten abweichen (Abschnitt 5.4).

Tabelle A.2: *Eingefrorene und adaptierte Köpfe beider Stränge auf der Referenzskala (Injektion, volle Validierungsmenge).*

Strang	Kopf	Eingang	mIoU bzw. mAP	NDS
Segmentierung	eingefroren	Vorhersage	0,5898	—
Segmentierung	eingefroren	real	0,6295	—
Segmentierung	adaptiert (Startwert 42)	Vorhersage	0,6012	—
Segmentierung	adaptiert (Startwert 42)	real	0,6293	—
Detektion	eingefroren	Vorhersage	0,3438	0,4761
Detektion	eingefroren	real	0,6858	0,7146
Detektion	adaptiert, Variante 1 (Startwert 42)	Vorhersage	0,4329	0,5328
Detektion	adaptiert, Variante 1 (Startwert 42)	real	0,6603	0,6918
Detektion	adaptiert, Variante 2 (Startwert 42)	Vorhersage	0,4292	0,5333
Detektion	adaptiert, Variante 2 (Startwert 42)	real	0,6710	0,7040

A.3 Einzelverläufe der Robustheitsprüfung

Die folgenden Abbildungen zeigen die Einzelverläufe hinter der Sammeldarstellung in Abbildung 5.4. Sie sind so zu lesen wie dort: Bezugsgröße ist jeweils die minimale Konfiguration, das graue Band markiert die Seed-Variabilität.

Abbildung A.1 zeigt die autoregressive Rollout-Auswertung über vier Schritte, getrennt nach Modell und naiver Persistenz. Die zugehörige Diskussion steht in Abschnitt 5.2. Abbildung A.2 stellt die vier Injektionsvarianten der Ego-Konditionierung gegenüber, Abbildung A.3 die naive und die ego-kompensierte Persistenz über den Rollout-Horizont. Abbildung A.4 und Abbildung A.5 zeigen die Kontext- und EMA-Varianten sowie Task-Loss, Kapazität und Residualvariante.

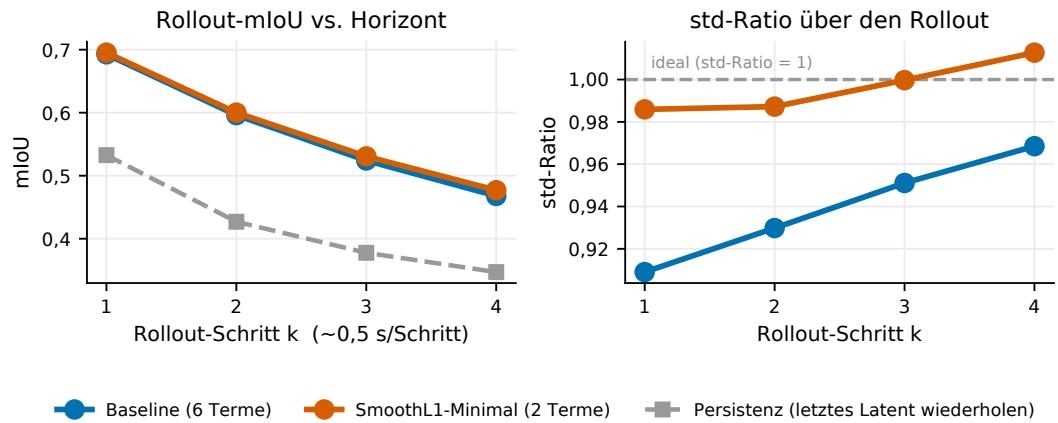


Abbildung A.1: Autoregressiver Rollout ($k = 1$ bis 4): mIoU über den Horizont für Modell und naive Persistenz (Vollvalidierung).

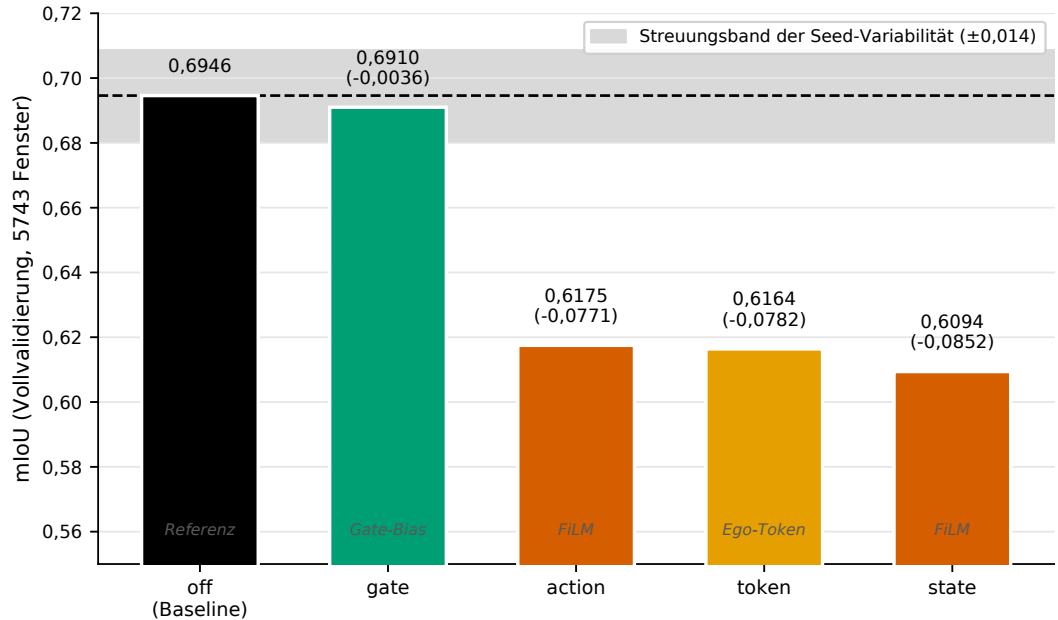


Abbildung A.2: Ego-Konditionierung: die vier Injektionsvarianten gegen die Referenz ohne Konditionierung.

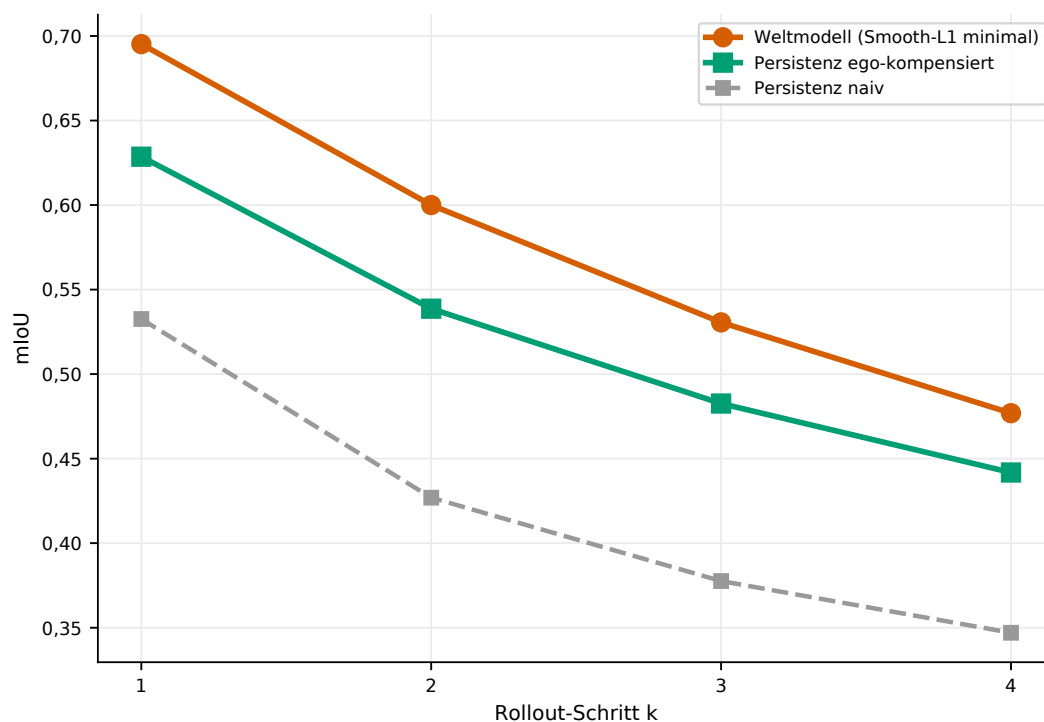


Abbildung A.3: Naive und ego-kompensierte Persistenz über den Rollout-Horizont im Vergleich zum Modell.

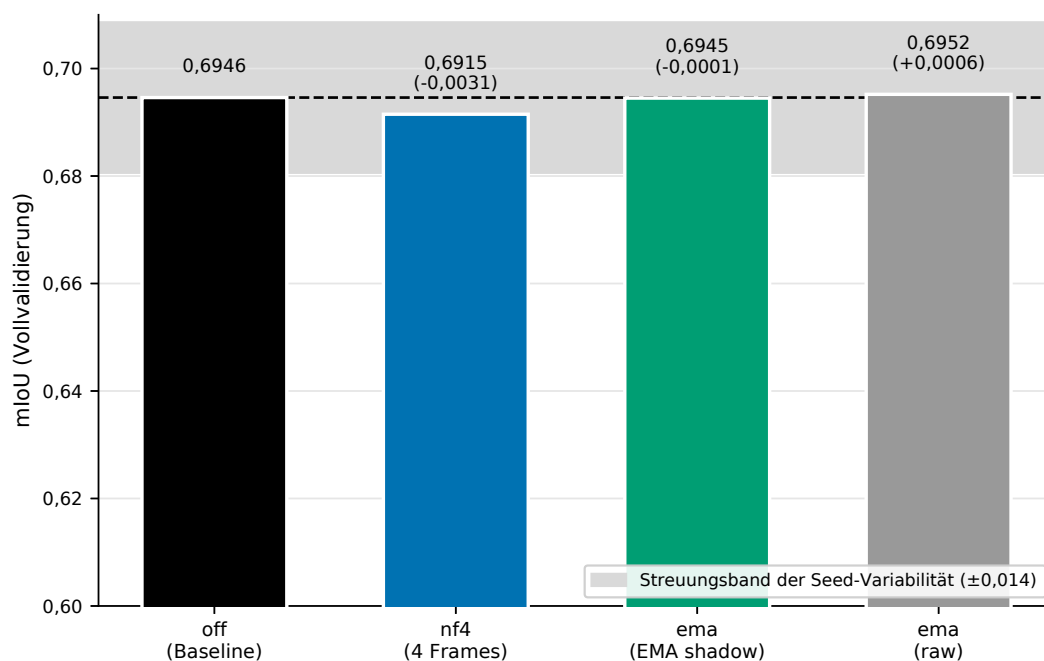


Abbildung A.4: Kontext- und EMA-Varianten gegen die Referenz.

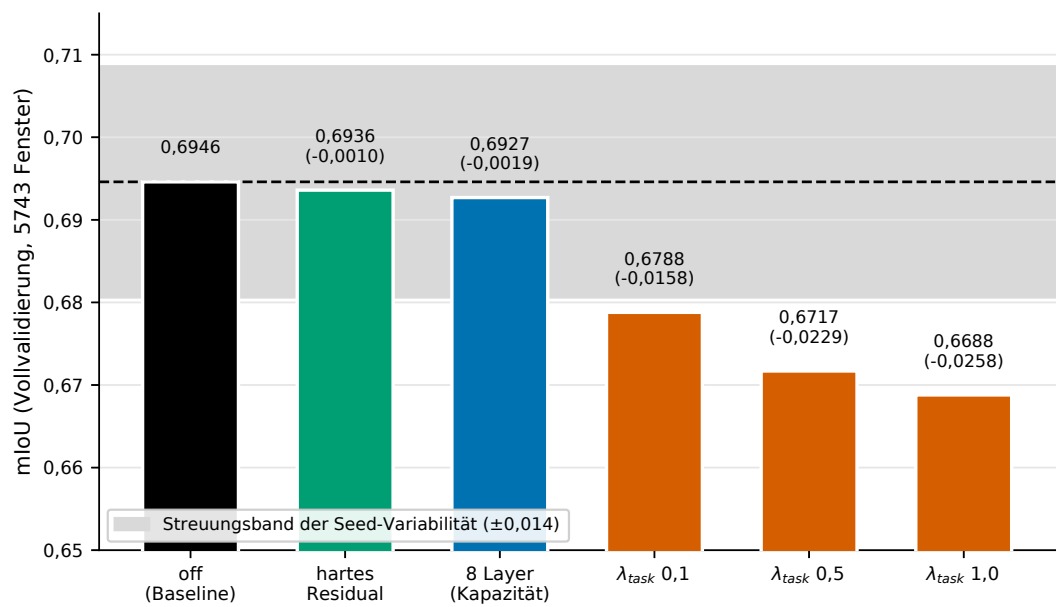


Abbildung A.5: *Decoder-Task-Loss (drei Gewichtungen), verdoppelte Tiefe und hartes Residual gegen die Referenz.*

A.4 Ergänzungen zu den generativen Köpfen

Ergänzend zu Abschnitt 5.3 zeigen Abbildung A.6 und Abbildung A.7 die CVAE-Darstellungen der beiden übrigen β -Werte. Die Ergebnisse bestätigen, dass sich die CVAE-Stichproben auch über drei Größenordnungen von β nur gering unterscheiden. Abbildung A.8 stellt den autoregressiven Rollout des deterministischen Modells und der Flow-Stichproben gegenüber. Dabei bleibt die Streuung des deterministischen Modells über die Schritte stabil, während die Abweichung der Flow-Stichproben mit dem Horizont zunimmt.

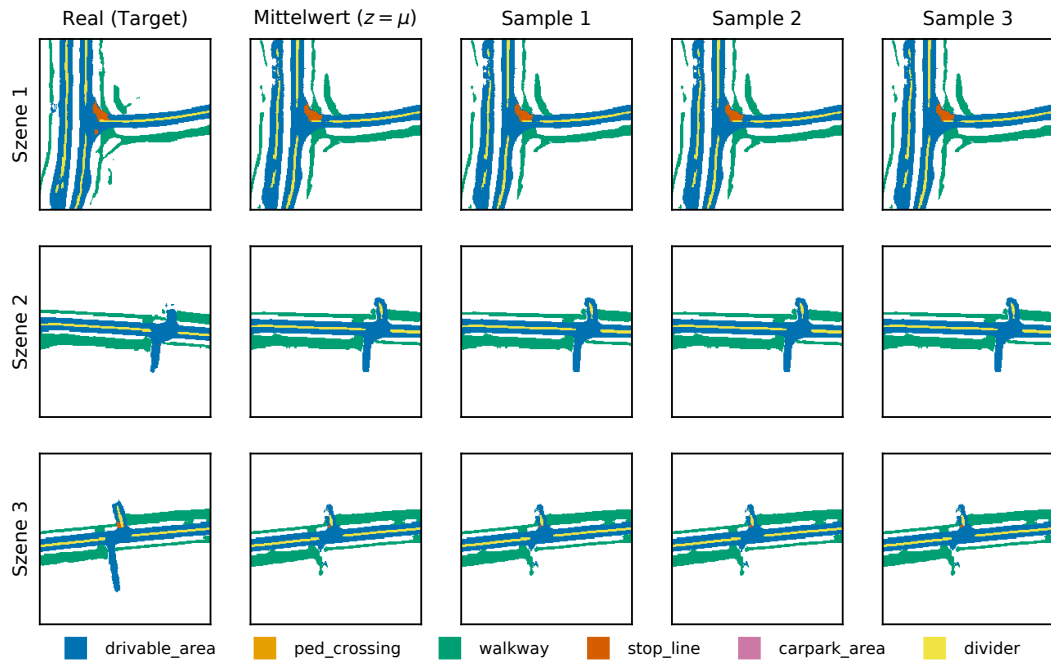


Abbildung A.6: CVAE-Darstellung für $\beta = 0,001$: Mittelwert und Prior-Stichproben.

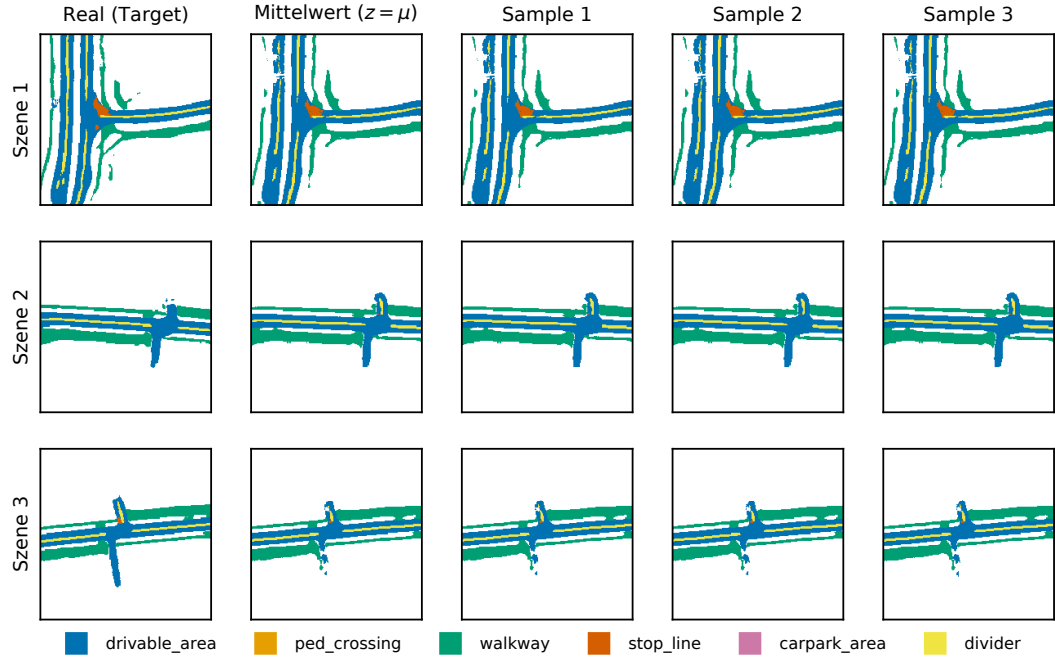


Abbildung A.7: CVAE-Darstellung für $\beta = 0,1$: Mittelwert und Prior-Stichproben.

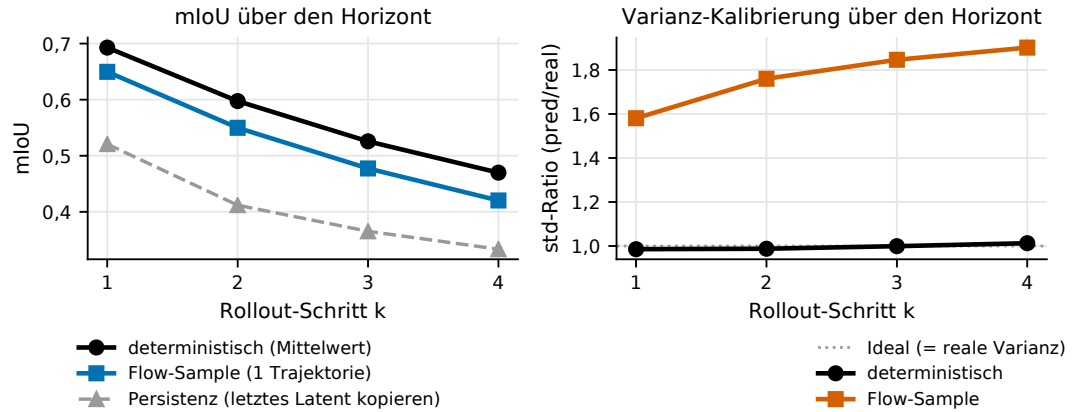


Abbildung A.8: Rollout-Vergleich: deterministisches Modell gegen Flow-Stichproben (*mIoU* und *Streuungsverhältnis* über den Horizont).

A.5 Qualitative Detektionspanels

Die folgenden Panels ergänzen Abschnitt 5.4 um weitere Szenen. Wie im Haupttext wurden die gezeigten Szenen nach maximaler Differenz zwischen den verglichenen Varianten ausgewählt, um die Unterschiede sichtbar zu machen. Sie sind nicht repräsentativ für den Durchschnittsfall. Abbildung A.9 zeigt Detektionen aus injizierten Bird's Eye View (BEV)-Repräsentationen des Weltmodells gegenüber der Persistenz, Abbildung A.10 den eingefrorenen gegen den adaptierten Kopf und Abbildung A.11 die beiden Fassungen des adaptierten Kopfes.

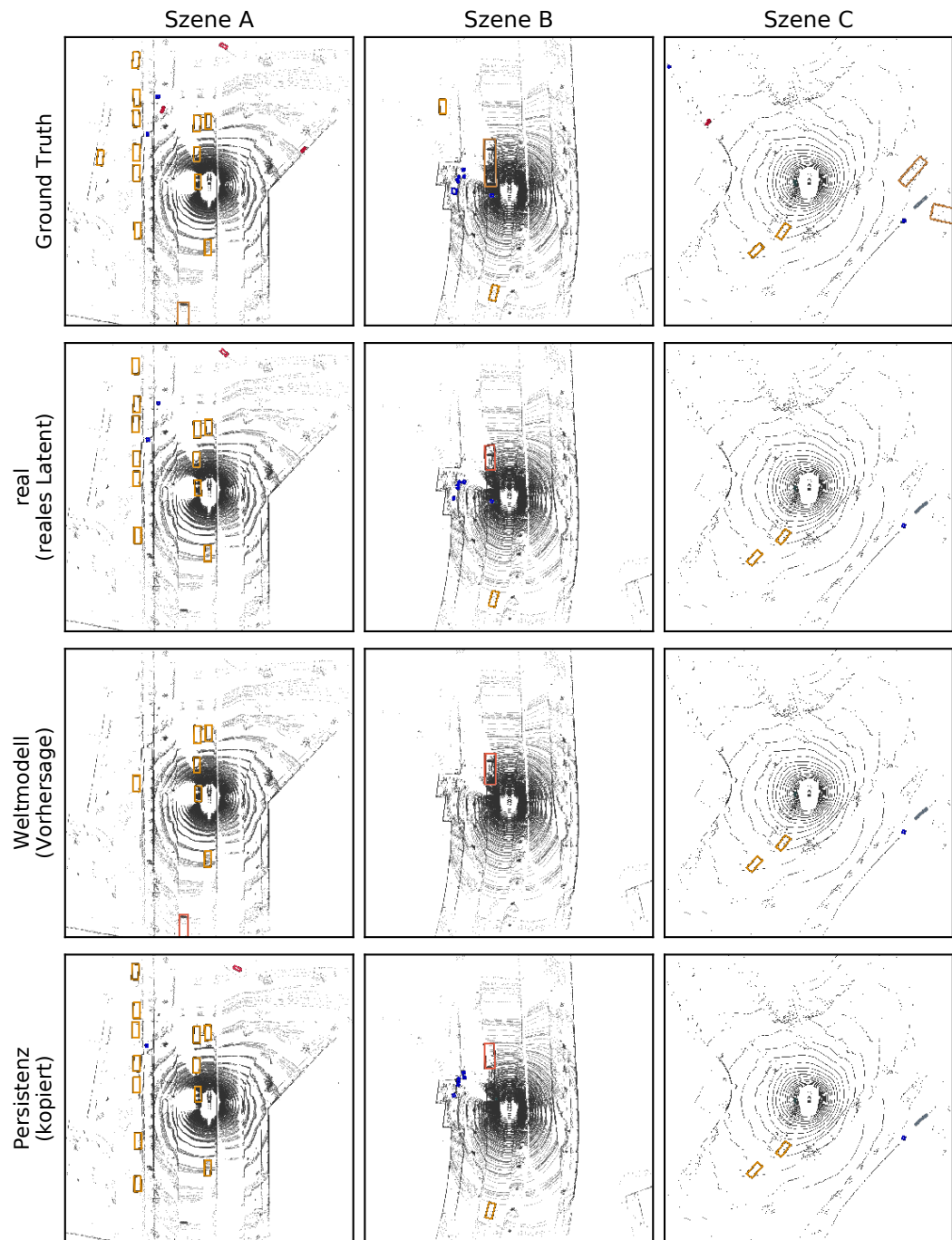


Abbildung A.9: 3D-Boxen aus injizierten BEV-Repräsentationen: Weltmodell gegen Persistenz (Szenenauswahl nach maximaler Differenz).

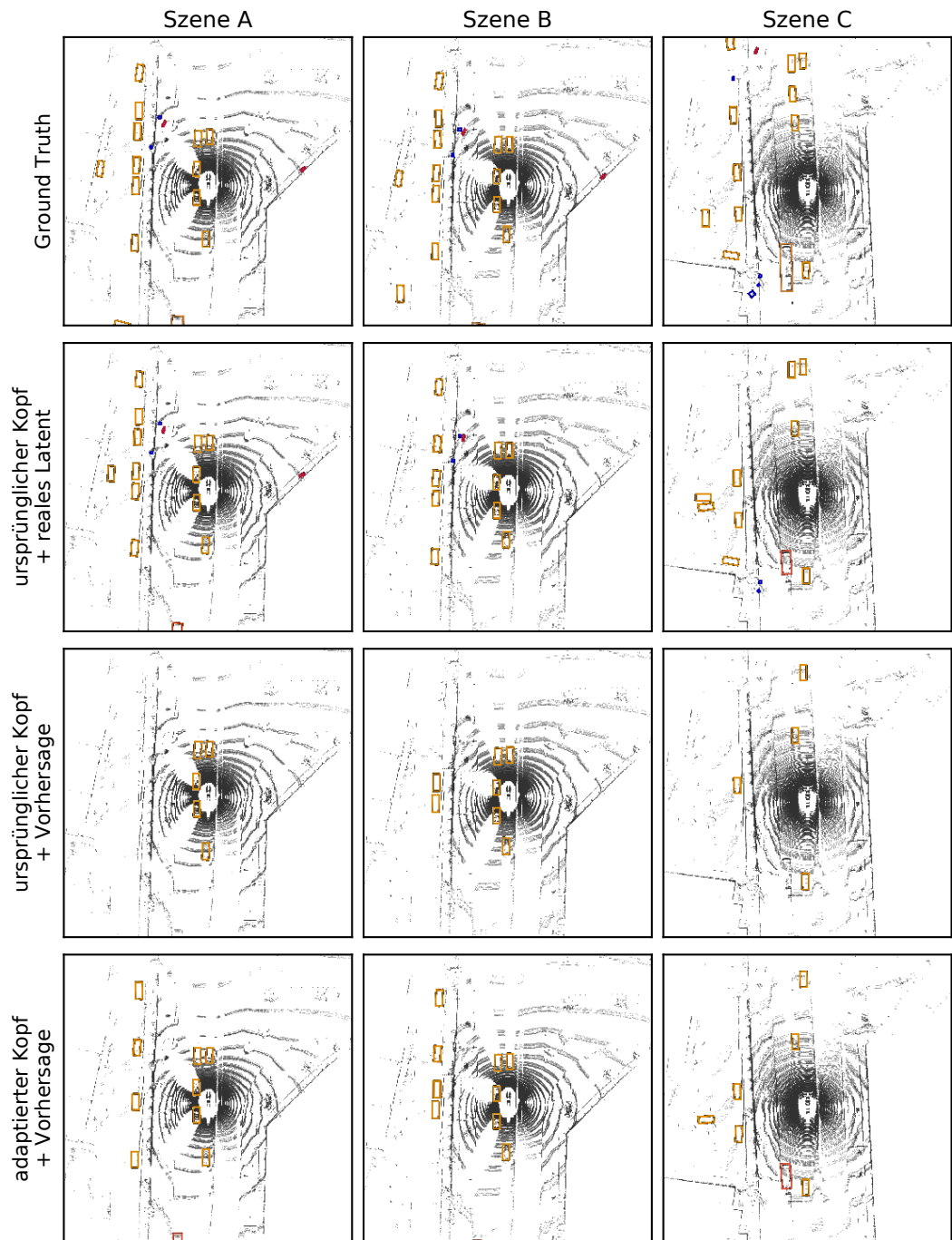


Abbildung A.10: Eingefrorener gegen adaptierten Detektionskopf auf identischen Vorhersagen (Szenenauswahl nach maximaler Differenz).

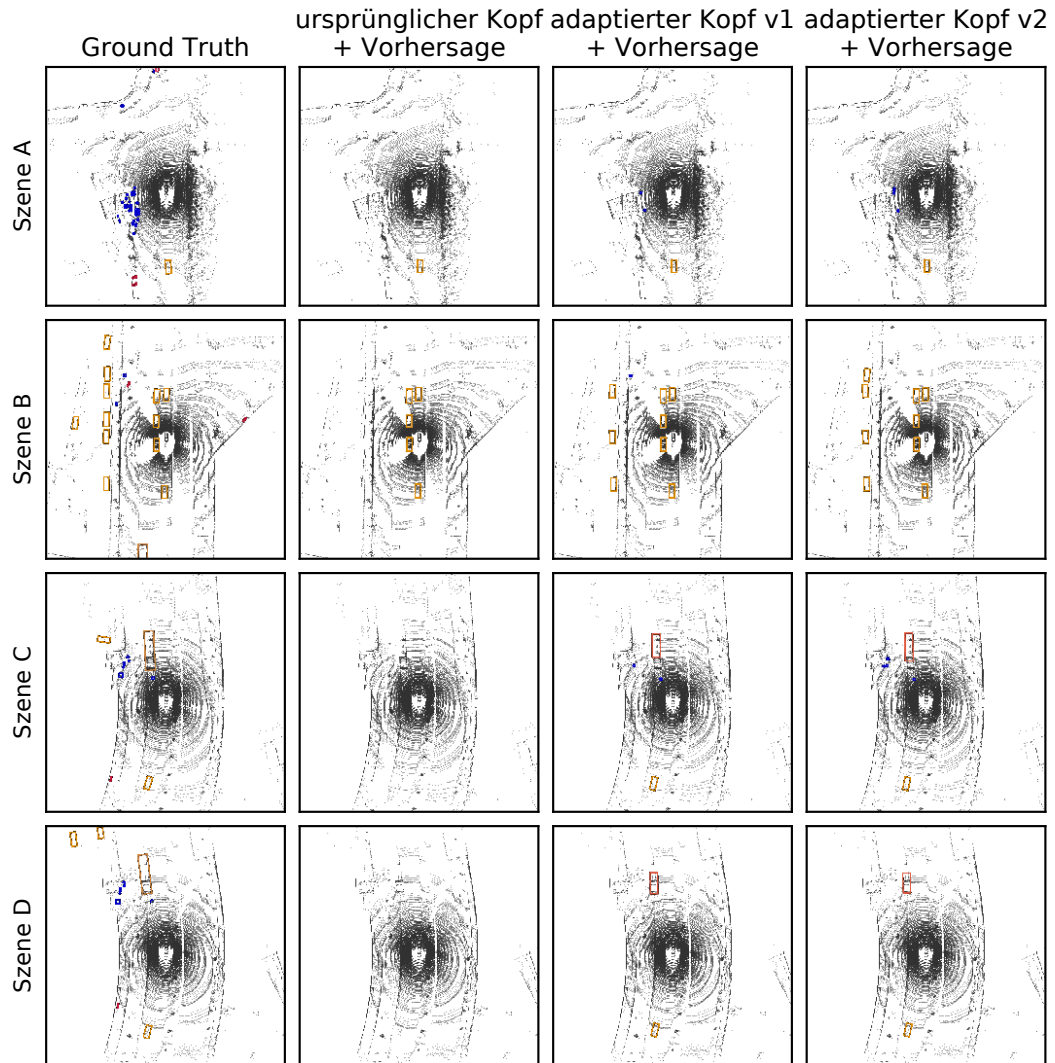


Abbildung A.11: Beide Fassungen des adaptierten Detektionskopfes im Vergleich (Szenenauswahl nach maximaler Differenz).

Literaturverzeichnis

- [1] T. Feng, W. Wang, and Y. Yang, "A Survey of World Models for Autonomous Driving," 2025, arXiv:2501.11260.
- [2] L. Russell, A. Hu, L. Bertoni, G. Fedoseev, J. Shotton, E. Arani, and G. Corrado, "GAIA-2: A Controllable Multi-View Generative World Model for Autonomous Driving," 2025, arXiv:2503.20523.
- [3] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. Rus, and S. Han, "BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 2774–2781.
- [4] A. J. Alrasheed, A. Yazdan Parast, B. Azam, J. Bailey, and N. Akhtar, "Latent Video Prediction Learns Better World Models," 2026, arXiv:2605.15618.
- [5] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, "LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels," 2026, arXiv:2603.19312.
- [6] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A Multimodal Dataset for Autonomous Driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11 621–11 631.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.
- [8] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.
- [9] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, "On Layer Normalization in the Transformer Architecture," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 10 524–10 533.
- [10] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014.
- [11] K. Sohn, H. Lee, and X. Yan, "Learning Structured Output Representation using Deep Conditional Generative Models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 3483–3491.
- [12] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating Sentences from a Continuous Space," in *Proc. SIGNLL Conf. Comput. Natural Lang. Learn. (CoNLL)*, 2016, pp. 10–21.
- [13] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 6840–6851.

- [14] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [15] G. Boduljak, Y. Lan, C. Rupprecht, and A. Vedaldi, “VFMF: World Modeling by Forecasting Vision Foundation Model Features,” 2025, arXiv:2512.11225.
- [16] D. P. Kingma and M. Welling, “An Introduction to Variational Autoencoders,” *Found. Trends Mach. Learn.*, vol. 12, no. 4, pp. 307–392, 2019.
- [17] J. Phillion and S. Fidler, “Lift, Splat, Shoot: Encoding Images From Arbitrary Camera Rigs by Implicitly Unprojecting to 3D,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 194–210.
- [18] S. Tu, X. Zhou, D. Liang, X. Jiang, Y. Zhang, X. Li, and X. Bai, “The Role of World Models in Shaping Autonomous Driving: A Comprehensive Survey,” 2025, arXiv:2502.10498.
- [19] A. Basant, A. Kar, D. Paschalidou, F. Wei, F. Ferroni, G. Garcia Cobo, H. Turki, H. Ling, J. Seo, J. Lucas, J. Z. Wu, J. Wang, J. Lorraine, J. Gao, K. He, K. Tothova, K. Xie, M. Tyszkiewicz, Q. Wu, R. de Lutio, R. Li, S. Fidler, S. W. Kim, T. Shen, T. Cao, T. Pfaff, W. Lew, X. Wu, X. Ren, Y. Lu, Y. Zhang, Z. Gojcic, and Z. Wang, “NVIDIA OmniDreams: Real-Time Generative World Model for Closed-Loop Autonomous Vehicle Simulation,” 2026, arXiv:2606.03159.
- [20] W. Zheng, W. Chen, Y. Huang, B. Zhang, Y. Duan, and J. Lu, “OccWorld: Learning a 3D Occupancy World Model for Autonomous Driving,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 55–72.
- [21] J. Mei, Y. Yang, X. Yang, L. Wen, J. Lv, B. Shi, and Y. Liu, “Vision-Centric 4D Occupancy Forecasting and Planning via Implicit Residual World Models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [22] Y. Li, L. Fan, J. He, Y. Wang, Y. Chen, Z. Zhang, and T. Tan, “Enhancing End-to-End Autonomous Driving with Latent World Model,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
- [23] Y. Li, Y. Wang, Y. Liu, J. He, L. Fan, and Z. Zhang, “End-to-End Driving with Online Trajectory Evaluation via BEV World Model,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 27 137–27 146.
- [24] Y. Zheng, P. Yang, Z. Xing, Q. Zhang, Y. Zheng, Y. Gao, P. Li, T. Zhang, Z. Xia, P. Jia, and D. Zhao, “World4Drive: End-to-End Autonomous Driving via Intention-aware Physical Latent World Model,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 28 632–28 642.
- [25] Y. Hong, X. Zhou, Y. Li, X. Zhou, L. Liu, Y. Luo, S. Xu, L. Yang, and Z. Song, “DriveFuture: Future-Aware Latent World Models for Autonomous Driving,” 2026, arXiv:2605.09701.
- [26] E. Karypidis, I. Kakogeorgiou, S. Gidaris, and N. Komodakis, “DINO-Foresight: Looking into the Future with DINO,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

- [27] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025, pp. 79 115–79 135.
- [28] T. Keressies, G. Berton, J. He, Q. Yu, W. Ma, D. de Geus, G. Dubbelman, and L.-C. Chen, "A Frame is Worth One Token: Efficient Generative World Modeling with Delta Tokens," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026, pp. 27 978–27 988.
- [29] E. Karypidis, I. Kakogeorgiou, S. Gidaris, and N. Komodakis, "Advancing Semantic Future Prediction through Multimodal Visual Sequence Transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 3793–3803.
- [30] J. Song, Y. Liu, and K. A. Skinner, "CRISP: A Spatiotemporal Camera-Radar Backbone for Driving via Forecasting-Based World-Model Pretraining," 2026, arXiv:2607.04541.
- [31] R. Sharifullin, B. Jiang, and K. X. Chew, "Diffusion Transformer World-Action Model for AV Scene Prediction," 2026, arXiv:2606.12987.
- [32] J. C. Walker, P. Vélez, L. Polania Cabrera, G. Zhou, S. Ebrahimi, R. Kabra, C. Doersch, M. Ovsjanikov, J. Carreira, and S. Ginosar, "Frozen Forecasting: A Unified Evaluation," 2025, arXiv:2507.13942.
- [33] L. Wang, Z. Yang, C. Bai, G. Zhang, X. Liu, X. Zheng, X.-X. Long, C.-T. Lu, and C. Lu, "Drive-JEPA: Video JEPA Meets Multimodal Trajectory Distillation for End-to-End Driving," 2026, arXiv:2601.22032.
- [34] H. Zhu and A. Choromanska, "Self-Supervised JEPA-based World Models for LiDAR Occupancy Completion and Forecasting," 2026, arXiv:2602.12540.
- [35] M. Fu, F. Feng, N. Hansen, and B. Huang, "Back to Parsimonious Latents: Learning Task-Centric World Models from Visual Foundations," 2026, arXiv:2605.25620.
- [36] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [37] E. Denton and R. Fergus, "Stochastic Video Generation with a Learned Prior," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1174–1183.
- [38] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual Reasoning with a General Conditioning Layer," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2018, pp. 3942–3951.
- [39] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, "Improved Variational Inference with Inverse Autoregressive Flow," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2016, pp. 4743–4751.

- [40] Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 6228–6237.